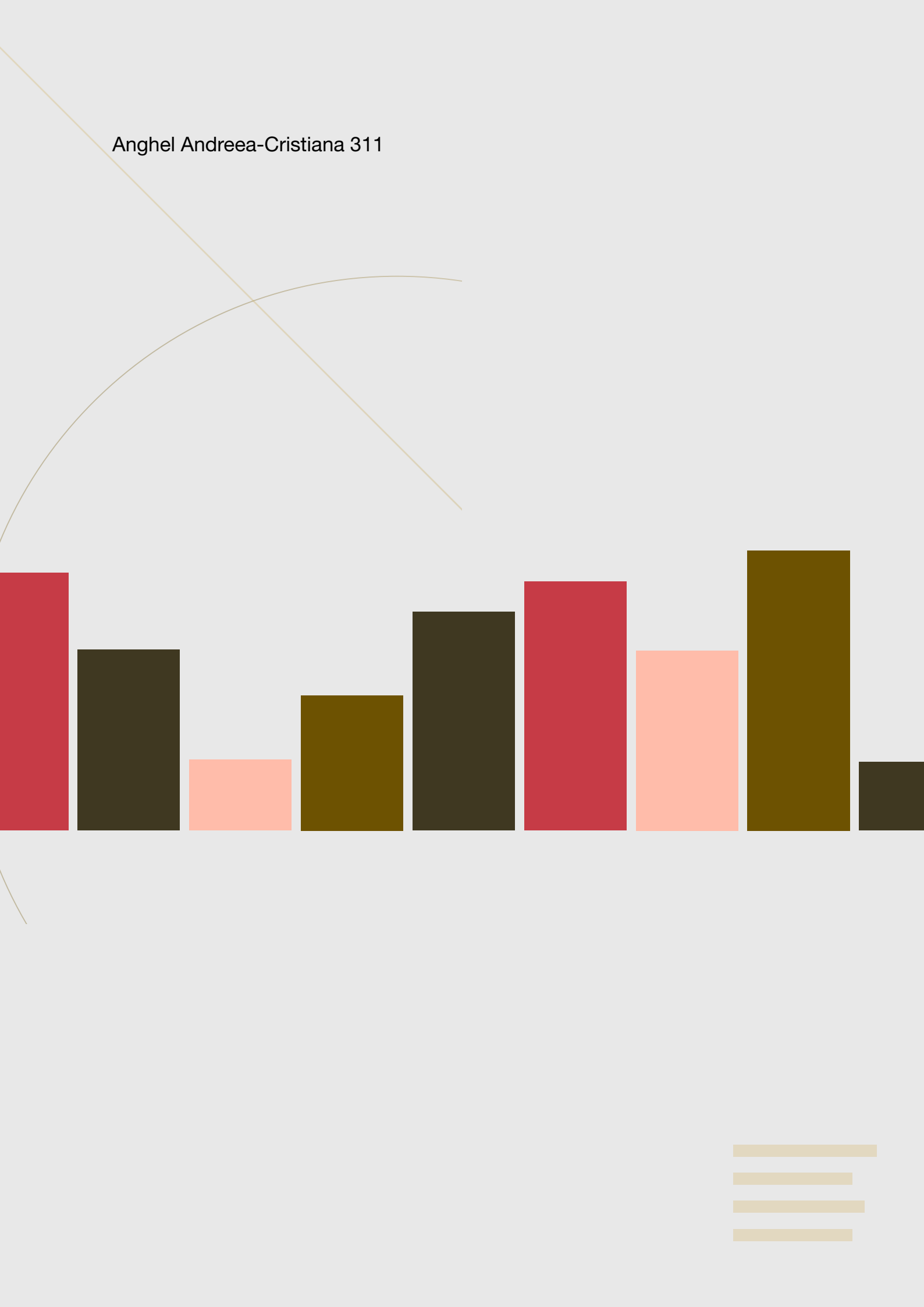




I. O inegalitate cu aplicații interdisciplinare

1. Inegalitatea lui Kantorovich-prezentare generală

Fie $A \in \mathbb{R}^{n \times n}$ o matrice simetrică pozitiv definită, cu spectrul inclus într-un interval finit:

$$0 < m = \lambda_{\min}(A) \leq \lambda_{\max}(A) = M < \infty.$$

Atunci, pentru orice vector $x \in \mathbb{R}^n$, $x \neq 0$, are loc inegalitatea:

$$(x^T A x) (x^T A^{-1} x) \leq \frac{(M + m)^2}{4Mm} (x^T x)^2.$$

În particular, pentru $\|x\| = 1$, $(x^T A x) (x^T A^{-1} x) \leq \frac{(M+m)^2}{4Mm}$.

2. Demonstrația probabilistică a lui T. W. Anderson(1971)

Deși inegalitatea este de natură algebrică, există o demonstrație elegantă, atribuită lui T. W. Anderson, care o reduce la o inegalitate despre momentele unei variabile aleatoare. Ideea-cheie a lui Anderson este următoarea: probabilitățile nu sunt introduse „din exterior”, ci sunt construite exact astfel încât formele pătratice să devină medii.

Această reinterpretare transformă o problemă de algebră liniară într-o problemă elementară de probabilități, în care singura informație folosită este faptul că o variabilă aleatoare este mărginită între două constante.

Pasul 1. Reducerea la cazul diagonal

A este o matrice simetrică pozitiv definită, deci există o matrice ortogonală Q și o matrice diagonală $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, $m = \lambda_1 \leq \dots \leq \lambda_n = M$, astfel încât $A = Q\Lambda Q^T$.

Notând cu $y = Q^T x$. Atunci:

$$x^T A x = \sum_{i=1}^n \lambda_i y_i^2, \quad x^T A^{-1} x = \sum_{i=1}^n \frac{1}{\lambda_i} y_i^2, \quad x^T x = \sum_{i=1}^n y_i^2.$$

Pasul 2. Construirea distribuției de probabilitate

Definim $p_i = \frac{y_i^2}{\sum_{j=1}^n y_j^2}$, $i = 1, \dots, n$.

Atunci $p_i \geq 0$ și $\sum_i p_i = 1$, deci (p_i) este o distribuție de probabilitate discretă.

Pasul 3. Introducerea variabilei aleatoare

Construim o variabilă aleatoare discretă T definite prin: $\mathbb{P}(T = \lambda_i) = p_i$. Atunci:

$$E(T) = \frac{x^T A x}{x^T x}, \quad E(1/T) = \frac{x^T A^{-1} x}{x^T x}.$$

Astfel, inegalitatea lui Kantorovich devine echivalentă cu:

$$E(T) E(1/T) \leq \frac{(M + m)^2}{4Mm}.$$

Pasul 4. Inegalitatea probabilistică elementară

Deoarece T ia valori doar în intervalul $[m, M]$, avem că $(T - m)(M - T) \geq 0$.

Aceasta conduce, printr-o manipulare algebrică elementară și aplicarea operatorului de medie, la ceea ce trebuia demonstrat.

Esența demonstrației este aceea că inegalitatea matricială este redusă la o inegalitate pentru o variabilă aleatoare pozitivă, folosind doar faptul că aceasta este mărginită între două constante. Această reformulare este contribuția conceptuală majoră a lui Anderson.

3. Cerințe de proiect – simulare și convergență

3.1 Reconstrucția demonstrației prin simulare

- 1) Alegeți $n \geq 6$ și un set de valori $m = \lambda_1 \leq \dots \leq \lambda_n = M$.
- 2) Generați un vector aleator $y = (y_1, \dots, y_n)$ și definiți $p_i = \frac{y_i^2}{\sum_j y_j^2}$.
- 3) Definiți variabila aleatoare T cu $\mathbb{P}(T = \lambda_i) = p_i$.
- 4) Simulați N observații din T și estimați $\hat{E}_N(T)$, $\hat{E}_N\left(\frac{1}{T}\right)$, $\hat{R}_N = \hat{E}_N(T)\hat{E}_N(1/T)$.
- 5) Comparați $\hat{R}_N$ cu limita teoretică $B = \frac{(M+m)^2}{4Mm}$.

Sugestii:

- pentru y_i : Normal(0,1), Student cu 3-5 grade de libertate, Laplace;
- studiați efectul raportului $\kappa = M/m$;
- măsurați „concentrarea” distribuției p (ex. $\max_i p_i$).

```
1 #-----3.1 RECONSTRUCTIA DEMONSTRATIEI PRIN SIMULARE-----
2 # 1) Alegem n >= 6 si setul de valori proprii (lambda)
3 n <- 10
4 m <- 2
5 M <- 20
6 lambdas <- seq(m, M, length.out = n)
7
8 # 2) Generam vectorul aleator y (ex: Normal(0,1)) si definim p_i
9 y <- rnorm(n, mean = 0, sd = 1)
10 p <- (y^2) / sum(y^2)
11
12 # 3) Definim variabila aleatoare T
13 # T ia valorile lambdas cu probabilitatile p
14
15 # 4) Simulam N observatii din T
16 N <- 10000
17 sim_T <- sample(lambdas, size = N, replace = TRUE, prob = p)
18
19 # Estimam mediile
20 E_T <- mean(sim_T) # Estimarea lui E(T)
21 E_inv_T <- mean(1/sim_T) # Estimarea lui E(1/T)
22 R_N <- E_T * E_inv_T # Produsul estimat
23
24 # 5) Comparam cu limita teoretica B
25 B <- ((M + m)^2) / (4 * M * m)
26
27 # Afisare rezultate
28 cat("Rezultatul estimat (R_N):", R_N, "\n")
29 cat("Limita teoretica (B):", B, "\n")
30 cat("Verificare (R_N <= B):", R_N <= B, "\n")
31
> # Afisare rezultate
> cat("Rezultatul estimat (R_N):", R_N, "\n")
Rezultatul estimat (R_N): 1.303603
> cat("Limita teoretica (B):", B, "\n")
Limita teoretica (B): 3.025
> cat("Verificare (R_N <= B):", R_N <= B, "\n")
Verificare (R_N <= B): TRUE
> |
```

3.2 Convergența estimatorilor

1) Pentru $N \in \{10^2, 10^3, 10^4\}$, folosind replicări independente, ilustrați numeric:

$$\hat{E}_N(T) \xrightarrow{\mathbb{P}} E(T), \quad \hat{E}_N(1/T) \xrightarrow{\mathbb{P}} E(1/T).$$

2) Argumentați (teoretic și prin simulare) că: $\hat{R}_N \xrightarrow{\mathbb{P}} R = E(T)E(1/T)$, deci $\hat{R}_N \xrightarrow{d} R$.

3) Explicați de ce pentru N mic pot apărea încălcări numerice ale inegalității, deși aceasta este adevărată pentru valorile teoretice.

```
33 #1
34 # Parametrii inițiali (din secțiunea 3.1)
35 n <- 10
36 lambdas <- seq(2, 20, length.out = n)
37 y <- rnorm(n)
38 p <- (y^2) / sum(y^2)
39
40 # Valorile teoretice (E(T) și E(1/T))
41 E_T_teoretic <- sum(p * lambdas)
42 E_inv_T_teoretic <- sum(p * (1/lambdas))
43
44 # Valorile N cerute
45 N_values <- c(100, 1000, 10000)
46
47 # Vectori pentru stocarea rezultatelor în vederea graficului
48 est_T <- c()
49 est_inv_T <- c()
50
51 cat("Valori teoretice: E(T) =", E_T_teoretic, " | E(1/T) =", E_inv_T_teoretic, "\n\n")
52
53 # Simularea convergenței
54 for (N in N_values) {
55   # Generăm eșantionul
56   sim_T <- sample(lambdas, size = N, replace = TRUE, prob = p)
57
58   # Estimăm valorile
59   E_N_T <- mean(sim_T)
60   E_N_inv_T <- mean(1/sim_T)
61
62   # Salvăm rezultatele
63   est_T <- c(est_T, E_N_T)
64   est_inv_T <- c(est_inv_T, E_N_inv_T)
65
66   # Afișăm rezultatele pentru a vedea convergența
67   cat("Pentru N =", N, ":\n")
68   cat("  Estimare E_N(T)      :", E_N_T, " (Eroare:", abs(E_N_T - E_T_teoretic), ")\n")
69   cat("  Estimare E_N(1/T)     :", E_N_inv_T, " (Eroare:", abs(E_N_inv_T - E_inv_T_teoretic), ")\n\n")
70 }
71
72 #3.2.2.
73 # 1. Configurare parametri (aceiași de la 3.1)
74 n <- 10
75 lambdas <- seq(2, 20, length.out = n)
76 y <- rnorm(n)
77 p <- (y^2) / sum(y^2)
78
79 # 2. Valoarea teoretică R
80 R_teoretic <- sum(p * lambdas) * sum(p * (1/lambdas))
81
82 # 3. Dimensiuni eșantion pentru testare
83 N_values <- c(10^2, 10^3, 10^4, 10^5)
84
85 cat("Valoarea țintă R =", R_teoretic, "\n\n")
86
87 for (N in N_values) {
88   # Generăm eșantionul din variabila T
89   sim_T <- sample(lambdas, size = N, replace = TRUE, prob = p)
90
91   # Calculăm estimatorii
92   E_N_T <- mean(sim_T)
93   E_N_inv_T <- mean(1/sim_T)
94
95   # Calculăm produsul estimat R_N
96   R_N <- E_N_T * E_N_inv_T
97
98   # Eroarea absolută
99   eroare <- abs(R_N - R_teoretic)
100
101   cat(sprintf("N = %-7d | R_N = %.6f | Eroare = %.6f\n", N, R_N, eroare))
102 }
103
> cat("Valori teoretice: E(T) =", E_T_teoretic, " | E(1/T) =", E_inv_T_teoretic, "\n\n")
Valori teoretice: E(T) = 10.87565 | E(1/T) = 0.1332504

Pentru N = 100 :
  Estimare E_N(T)      : 10.76 (Eroare: 0.1156496 )
  Estimare E_N(1/T)     : 0.1283671 (Eroare: 0.004883311 )

Pentru N = 1000 :
  Estimare E_N(T)      : 10.79 (Eroare: 0.08564962 )
  Estimare E_N(1/T)     : 0.1368391 (Eroare: 0.003588712 )

Pentru N = 10000 :
  Estimare E_N(T)      : 10.8728 (Eroare: 0.002849623 )
  Estimare E_N(1/T)     : 0.1327615 (Eroare: 0.0004888669 )

> cat("Valoarea tinta R =", R_teoretic, "\n\n")
Valoarea tinta R = 1.482935

+ }
N = 100      | R_N = 1.584673 | Eroare = 0.101738
N = 1000     | R_N = 1.445767 | Eroare = 0.037169
N = 10000    | R_N = 1.482310 | Eroare = 0.000626
N = 100000   | R_N = 1.483716 | Eroare = 0.000780
>
```


Convergența estimatorilor

Fie $(\lambda_i)_{i=1}^n$ valorile proprii ale matricei A cu distribuția de probabilitate asociată $p_i = \frac{\lambda_i^2}{\sum_{j=1}^n \lambda_j^2}$. Considerăm variabilă aleatoare discretă T definită prin $P(T=\lambda_i) = p_i$.

1. Argumentare teoretică a convergenței

Definim momente teoretice (momente) ca fiind:

$$E(T) = \sum_{i=1}^n \lambda_i p_i, \quad E\left(\frac{1}{T}\right) = \sum_{i=1}^n \frac{1}{\lambda_i} p_i$$

Fie $\{T_1, \dots, T_N\}$ un eșantion de N variabile aleatoare independente și identic distribuite (i.i.d) extrase din distribuția lui T . Estimătorii de selecție sunt definiți ca:

$$\hat{E}_N(T) = \frac{1}{N} \sum_{j=1}^N T_j, \quad \hat{E}_N\left(\frac{1}{T}\right) = \frac{1}{N} \sum_{j=1}^N \frac{1}{T_j}$$

Conform Legii Slabă a Numerelor Mari, deoarece $E(T) < \infty$ și $E\left(\frac{1}{T}\right) < \infty$ avem următoarea convergență în probabilitate:

$$\hat{E}_N(T) \xrightarrow{P} E(T), \quad \hat{E}_N\left(\frac{1}{T}\right) \xrightarrow{P} E\left(\frac{1}{T}\right)$$

Considerăm funcția de produs $g: \mathbb{R}^2 \rightarrow \mathbb{R}$, $g(u, v) = u \cdot v$. Deoarece g este o funcție continuă, aplicăm Teorema de Continuitate pentru convergență produsului:

$$\hat{R}_N = g(\hat{E}_N(T), \hat{E}_N\left(\frac{1}{T}\right)) \xrightarrow{P} g(E(T), E\left(\frac{1}{T}\right))$$

$$\hat{R}_N \xrightarrow{P} R, \text{ unde } R = E(T) \cdot E\left(\frac{1}{T}\right)$$

Deoarece convergența în probabilitate către o constantă implică convergența în distribuție, rezultă

$$\hat{R}_N \xrightarrow{d} R$$

argumentare
teoretică

```

104 #3.2.3.
105 # 1) Parametrii de bază
106 n <- 10
107 m <- 2
108 M <- 20
109 lambdas <- seq(m, M, length.out = n)
110
111 # 2) Generăm p_i (distribuție Normală)
112 set.seed(42)
113 y <- rnorm(n)
114 p <- (y^2) / sum(y^2)
115
116 # 3) Limita teoretică B
117 B <- ((M + m)^2) / (4 * M * m)
118
119 # 4) Funcție simplă pentru calculul R_N
120 simulate_RN <- function(N, replicari = 100) {
121   rezultate <- replicate(replicari, {
122     sim_T <- sample(lambdas, size = N, replace = TRUE, prob = p)
123     mean(sim_T) * mean(1/sim_T)
124   })
125   return(rezultate)
126 }
127
128 # 5) Colectăm date pentru N = 10^2, 10^3, 10^4
129 date_grafic <- list(
130   "N=100" = simulate_RN(100),
131   "N=1000" = simulate_RN(1000),
132   "N=10000" = simulate_RN(10000)
133 )
134

```

$$\text{Fie } R = E(T) \cdot E\left(\frac{1}{T}\right) \text{ și } \hat{R}_N = \hat{E}_N(T) \cdot \hat{E}_N\left(\frac{1}{T}\right)$$

$$R \leq B$$

$$\hat{E}_N(T) = \frac{1}{N} \sum_{i=1}^N T_i = E(T) + \varepsilon_1, \text{ unde } \varepsilon_1 \sim N\left(0, \frac{\sigma_T^2}{N}\right)$$

$$\hat{E}_N\left(\frac{1}{T}\right) = \frac{1}{N} \sum_{i=1}^N \frac{1}{T_i} = E\left(\frac{1}{T}\right) + \varepsilon_2, \text{ unde } \varepsilon_2 \sim N\left(0, \frac{\sigma_{\frac{1}{T}}^2}{N}\right)$$

$$\hat{R}_N = (E(T) + \varepsilon_1) \left(E\left(\frac{1}{T}\right) + \varepsilon_2 \right) = R + E(T)\varepsilon_2 + E\left(\frac{1}{T}\right)\varepsilon_1 + \varepsilon_1\varepsilon_2$$

$$N \text{ mic} \rightarrow \text{Var}(\Delta_N) \gg 0 \Rightarrow P(\Delta_N > B - R) > 0$$

$$\hat{R}_N > B \Leftrightarrow \Delta_N > B - R$$

$$\lim_{N \rightarrow \infty} P(\hat{R}_N > B) = 0$$

4. Aplicație practică: media aritmetică vs. media armonică

Fie T o variabilă aleatoare pozitivă cu valori în $[m, M]$. Definim media aritmetică $A = E(T)$ și respectiv media armonică $H = 1/E(1/T)$.

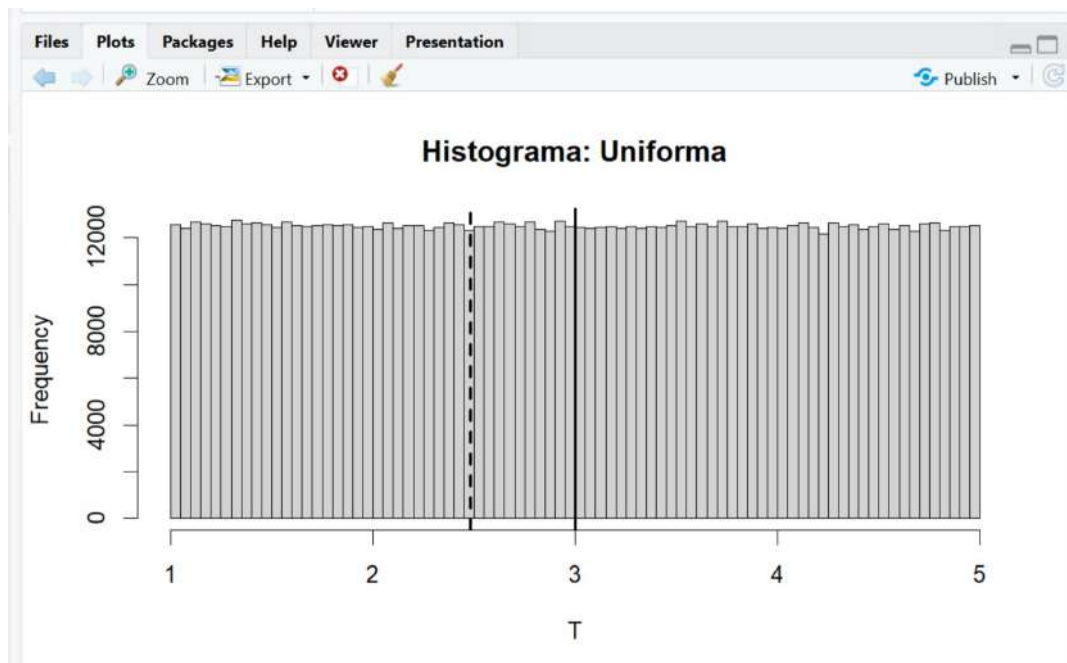
Raportul $\frac{A}{H} = E(T)E(1/T)$ este o măsură a heterogenității valorilor lui T .

Cerințe:

- 1) Construiți cel puțin patru distribuții diferite pentru T pe $[m, M]$.
- 2) Pentru fiecare distribuție estimați A , H , A/H .
- 3) Verificați numeric limita Kantorovich și interpretați diferențele între distribuții.
- 4) Explicați de ce două distribuții cu același interval $[m, M]$ pot avea heterogenități foarte diferite, dar toate respectă aceeași limită universală.

Distribuțiile recomandate sunt: uniformă pe $[m, M]$, Beta(α, β) re-scalată (pentru parametri diferiți), distribuție bimodală (amestec de două uniforme sau două Beta), distribuție discretă cu un număr finit de valori.

```
140 # 4 distributii pe [m, M]
141 # Estimam A = E(T), H = 1/E(1/T), A/H si verificam limita Kantorovich
142 # -----
143
144 # Intervalul [m, M]
145 m <- 1
146 M <- 5
147
148 #Numar de simulari
149 N <- 10^6
150
151 # Limita Kantorovich
152 B <- (M + m)^2 / (4 * M * m)
153
154 cat("Interval [m, M] = [", m, ", ", M, "]\n")
155 cat("Limita Kantorovich B =", B, "\n\n")
156
157
158 # 1) Distributia Uniforma pe [m, M]
159
160 T1 <- runif(N, min = m, max = M)
161
162 A1 <- mean(T1)
163 H1 <- 1 / mean(1 / T1)
164 AH1 <- A1 / H1
165
166 cat("1) Uniforma[m, M]\n")
167 cat("A =", A1, "\n")
168 cat("H =", H1, "\n")
169 cat("A/H =", AH1, "\n")
170 cat("Respecta limita? ", AH1 <= B, "\n\n")
171
172 hist(T1, breaks=60, main="Histograma: Uniforma", xlab="T", col="lightgray")
173 abline(v=A1, lwd=2) # linie la media aritmetica
174 abline(v=H1, lwd=2, lty=2) # linie la media armonica
175
176
> cat("Interval [m, M] = [", m, ", ", M, "]\n")
Interval [m, M] = [ 1 , 5 ]
> cat("Limita Kantorovich B =", B, "\n\n")
Limita Kantorovich B = 1.8
>
> cat("1) Uniforma[m, M]\n")
1) Uniforma[m, M]
> cat("A =", A1, "\n")
A = 3.000828
> cat("H =", H1, "\n")
H = 2.486086
> cat("A/H =", AH1, "\n")
A/H = 1.207049
> cat("Respecta limita? ", AH1 <= B, "\n\n")
Respecta limita? TRUE
```



```

177 # 2) Distributia Beta(alpha, beta) re-scalata pe [m, M]
178
179 alpha <- 2
180 beta <- 2
181
182 U2 <- rbeta(N, shape1 = alpha, shape2 = beta) # in [0,1]
183 T2 <- m + (M - m) * U2 # in [m,M]
184
185 A2 <- mean(T2)
186 H2 <- 1 / mean(1 / T2)
187 AH2 <- A2 / H2
188
189 cat("2) Beta(", alpha, ",", beta, ") re-scalata pe [m, M]\n", sep = "")
190 cat("A =", A2, "\n")
191 cat("H =", H2, "\n")
192 cat("A/H =", AH2, "\n")
193 cat("Respecta limita? ", AH2 <= B, "\n\n")
194
195 hist(T2, breaks=60, main="Histograma: Beta rescalata", xlab="T", col="lightgray")
196 abline(v=A2, lwd=2)
197 abline(v=H2, lwd=2, lty=2)
198

```

```

> cat("2) Beta(", alpha, ",", beta, ") re-scalata pe [m, M]\n", sep = "")
2) Beta(2,2) re-scalata pe [m, M]
> cat("A =", A2, "\n")
A = 2.999271
> cat("H =", H2, "\n")
H = 2.697084
> cat("A/H =", AH2, "\n")
A/H = 1.112042
> cat("Respecta limita? ", AH2 <= B, "\n\n")
Respecta limita? TRUE

```




```

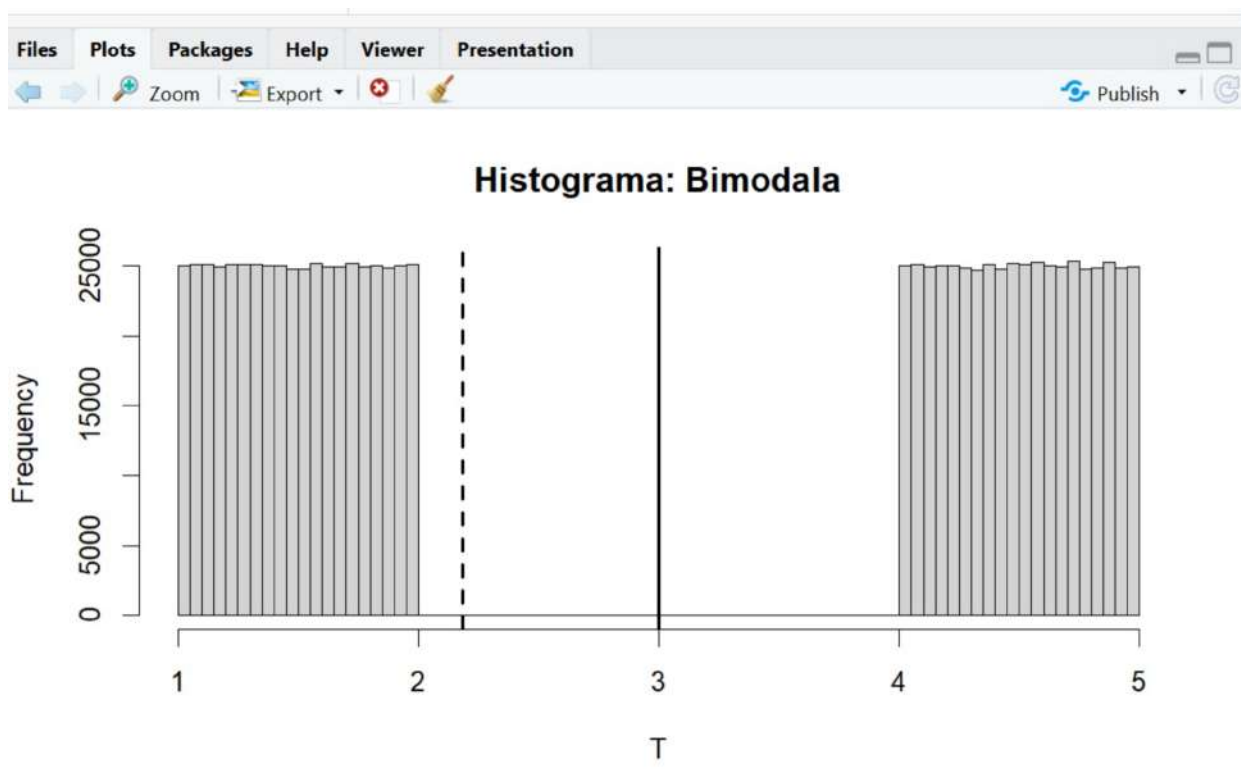
200 # 3) Distribuție Bimodală: amestec de două uniforme
201 # Distribuția bimodală a fost construită generând jumătate dintre observații dintr-o
202 # distribuție uniformă pe un subinterval inferior și jumătate dintr-o distribuție uniformă
203 # pe un subinterval superior, obținând astfel o concentrare a valorilor în jurul extremităților
204 # intervalului.
205
206
207 # Prima jumătate: valori mici
208 T3a <- runif(N/2, min = m, max = 2)
209
210 # A doua jumătate: valori mari
211 T3b <- runif(N/2, min = 4, max = M)
212
213 # Le combinăm
214 T3 <- c(T3a, T3b)
215
216 # Le amestecăm
217 T3 <- sample(T3)
218
219 # Calcule
220 A3 <- mean(T3)
221 H3 <- 1 / mean(1 / T3)
222 AH3 <- A3 / H3
223
224 cat("3) Bimodală\n")
225 cat("A =", A3, "\n")
226 cat("H =", H3, "\n")
227 cat("A/H =", AH3, "\n")
228 cat("Respecta limita? ", AH3 <= B, "\n\n")
229
230 hist(T3, breaks=60, main="Histograma: Bimodală", xlab="T", col="lightgray")
231 abline(v=A3, lwd=2)
232 abline(v=H3, lwd=2, lty=2)
233
234 ..

```

```

> cat("3) Bimodală\n")
3) Bimodală
> cat("A =", A3, "\n")
A = 3.000322
> cat("H =", H3, "\n")
H = 2.182866
> cat("A/H =", AH3, "\n")
A/H = 1.374487
> cat("Respecta limita? ", AH3 <= B, "\n\n")
Respecta limita? TRUE

```




```

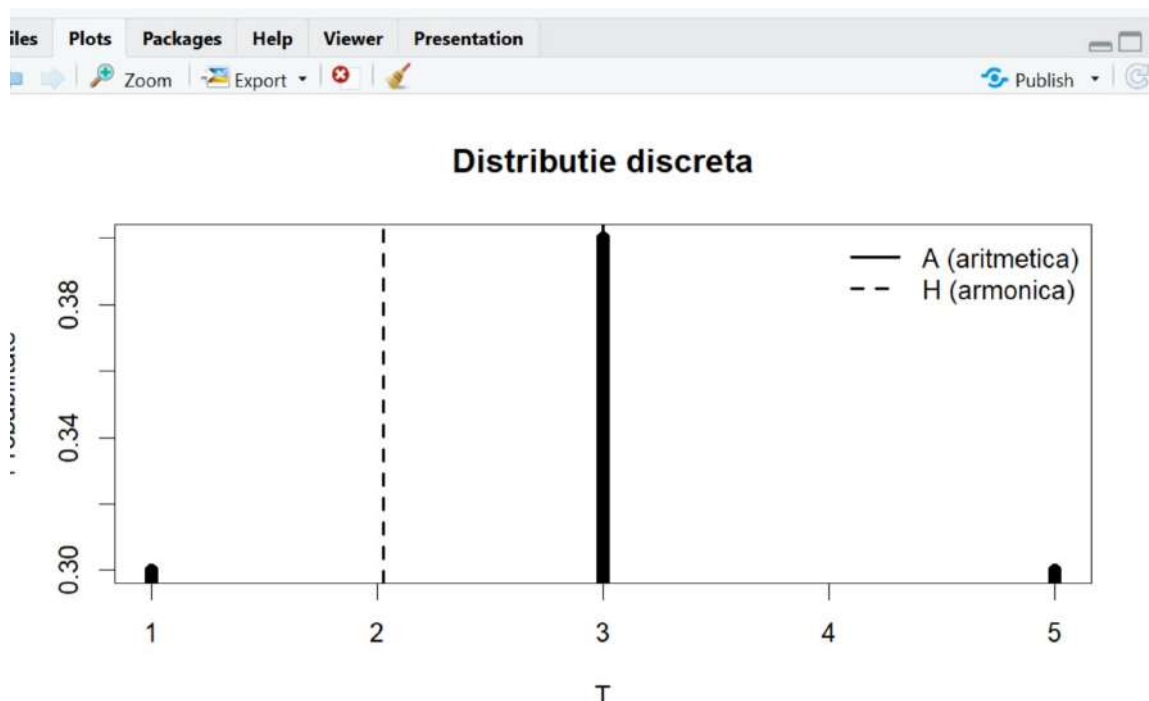
235 # -----
236 # 4) Distribuție Discretă: valori finite
237 #   Valori: {m, (m+M)/2, M} cu probabilități {0.3, 0.4, 0.3}
238 # -----
239
240 #Distribuția discretă a fost construită considerând un număr finit de valori posibile în
241 #intervalul [m,M], fiecare asociată cu o probabilitate dată.
242 #Prezența unei probabilități semnificative pentru valorile mici conduce la o scădere a
243 #mediei armonice și, implicit, la creșterea raportului A/H.
244
245 valori <- c(m, (m + M) / 2, M)
246 prob <- c(0.3, 0.4, 0.3)
247
248 T4 <- sample(valori, size = N, replace = TRUE, prob = prob)
249
250 A4 <- mean(T4)
251 H4 <- 1 / mean(1 / T4)
252 AH4 <- A4 / H4
253
254 cat("4) Discretă (3 valori)\n")
255 cat("A =", A4, "\n")
256 cat("H =", H4, "\n")
257 cat("A/H =", AH4, "\n")
258 cat("Respecta limita? ", AH4 <= B, "\n\n")
259
260 plot(valori, prob, type = "h", lwd = 8,
261      main = "Distribuție discretă",
262      xlab = "T",
263      ylab = "Probabilitate")
264
265

```

```

> cat("4) Discretă (3 valori)\n")
4) Discretă (3 valori)
> cat("A =", A4, "\n")
A = 10.96949
> cat("H =", H4, "\n")
H = 4.942313
> cat("A/H =", AH4, "\n")
A/H = 2.219505
> cat("Respecta limita? ", AH4 <= B, "\n\n")
Respecta limita? TRUE

```



```

267 AH <- c(AH1, AH2, AH3, AH4)
268 nume <- c("Uniforma", "Beta", "Bimodala", "Discreta")
269
270 barplot(AH, names.arg=nume, main="Comparatie heterogenitate (A/H)",
271         ylab="A/H", col="lightgray")
272
273 abline(h=B, lwd=2) # linia limitei Kantorovich
274
275 #Graficul comparativ al raportului A/H evidențiază diferențe semnificative de heterogenitate
276 #între distribuțiile analizate, deși toate sunt definite pe același interval [m,M].
277 #Distribuția Beta rescalată prezintă cea mai mică heterogenitate, deoarece valorile sunt
278 #concentrate în jurul mediei și există puține valori mici.
279 #Distribuția uniformă are o heterogenitate intermediară, întrucât valorile sunt distribuite
280 #egal pe interval.
281 #Distribuția bimodală este mai eterogenă, deoarece valorile sunt concentrate
282 #în apropierea extremităților intervalului.
283 #Cea mai mare heterogenitate este observată în cazul distribuției discrete, unde probabilitatea
284 #semnificativă asociată valorii minime conduce la o scădere accentuată a mediei armonice.
285
286 cat("\n--- Verificare Kantorovich ---\n")
287 cat("B (limita Kantorovich) =", B, "\n\n")
288
289 cat("Uniforma: A =", A1, " H =", H1, " A/H =", AH1, "\n")
290 cat("Beta:      A =", A2, " H =", H2, " A/H =", AH2, "\n")
291 cat("Bimodala: A =", A3, " H =", H3, " A/H =", AH3, "\n")
292 cat("Discreta: A =", A4, " H =", H4, " A/H =", AH4, "\n")
293
294
295 # 1) A (media aritmetica) depinde de valorile lui T.
296 # 2) H (media armonica) este foarte sensibila la valori mici ale lui T,
297 #    deoarece 1/T devine mare cand T este aproape de m.
298 # 3) De aceea, distributiile care au mai multe valori mici (aproape de m)
299 #    au H mai mica si raportul A/H mai mare (heterogenitate mai mare).
300 # 4) Chiar daca distributiile sunt diferite in interiorul intervalului [m,M],
301 #    toate respecta aceeasi limita universala Kantorovich B, care depinde doar de m si M.
302

```

```

--- Verificare Kantorovich ---

```

```

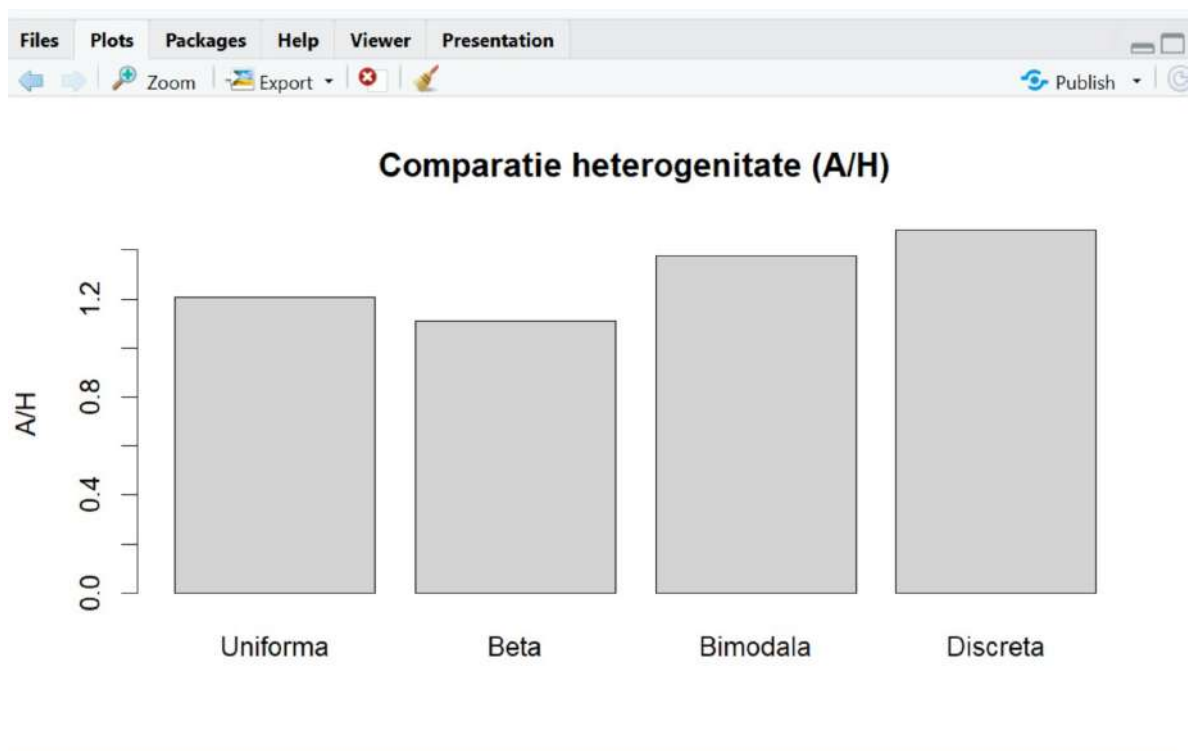
> cat("B (limita Kantorovich) =", B, "\n\n")
B (limita Kantorovich) = 1.8

```

```

>
> cat("Uniforma: A =", A1, " H =", H1, " A/H =", AH1, "\n")
Uniforma: A = 3.000828 H = 2.486086 A/H = 1.207049
> cat("Beta:      A =", A2, " H =", H2, " A/H =", AH2, "\n")
Beta:      A = 2.999271 H = 2.697084 A/H = 1.112042
> cat("Bimodala: A =", A3, " H =", H3, " A/H =", AH3, "\n")
Bimodala: A = 3.000322 H = 2.182866 A/H = 1.374487
> cat("Discreta: A =", A4, " H =", H4, " A/H =", AH4, "\n")
Discreta: A = 3.000492 H = 2.028274 A/H = 1.479333

```



II. Familia exponențială, estimare și acurătatea aproximării normale (Berry-Esseen)

Def. O variabilă aleatoare X face parte din **familia exponențială s-dimensională** dacă densitatea/funcția sa de masă poate fi scrisă sub forma:

$$f_X(x | \theta) = h(x) \exp(\sum_{i=1}^s \eta_i(\theta) T_i(x) - A(\theta)), \text{ unde:}$$

- θ reprezintă parametrul (sau vectorul de parametri),
- $T_1, \dots, T_s$ sunt statistici suficiente,
- $\eta_1, \dots, \eta_s$ sunt funcții reale de θ ,
- $A(\theta)$ este constanta de log-normalizare, al cărei rol este de a asigura că funcția definită este o densitate/funcție de masă de probabilitate.

Exemplu: Fie $X \sim \mathcal{N}(\mu, \sigma^2)$. Densitatea sa este $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$.

Aceasta poate fi rescrisă sub forma $f(x) = \exp\left(\frac{\mu}{\sigma^2}x - \frac{1}{2\sigma^2}x^2 - \left(\frac{\mu^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma^2)\right)\right)$.

Se identifică statisticile suficiente $T_1(x) = x$, $T_2(x) = x^2$, de unde rezultă că repartiția normală aparține familiei exponențiale 2-dimensionale.

Cerințe

1. Apartenența la familia exponențială și reprezentări grafice

a) Verificați dacă fac parte din familia exponențială următoarele repartiții: Binomială $\text{Bin}(n, p)$, Geometrică $\text{Geom}(p)$, Poisson $\text{Pois}(\lambda)$, Exponențială $\text{Exp}(\lambda)$, Gamma $\Gamma(\alpha, \beta)$, Beta $\text{Beta}(\alpha, \beta)$, Uniformă $U(a, b)$. În caz afirmativ, precizați dimensiunea familiei și statisticile suficiente.

b) Ilustrați grafic, în R, funcțiile de masă/densitățile repartițiilor de mai sus pentru 4 seturi de parametri particaliari, la alegere, în cadrul aceluiași sistem de axe ortogonale (fiecare repartiție va avea reprezentări grafice distincte de celelalte).

II Familia exponențială, estimare și acurătatea aproximării

1. Apartenența la familia exponențială și reprezentări grafice

a) $X \sim \text{Bin}(n, p)$ $p \in (0, 1)$, $n \in \mathbb{N}$ cunoscut

$$P(X=x) = C_n^x p^x (1-p)^{n-x}, \quad x=0, \dots, n$$

$$\begin{aligned} P(X=x) &= C_n^x \cdot e^{\ln[p^x \cdot (1-p)^{n-x}]} = C_n^x \cdot e^{x \ln p + (n-x) \ln(1-p)} \\ &= C_n^x \cdot e^{x \ln p + n \ln(1-p) - x \ln(1-p)} \\ &= C_n^x \cdot e^{x \ln \frac{p}{1-p} + n \ln(1-p)} = C_n^x \cdot e^{x \ln \frac{p}{1-p} - (-n \ln(1-p))} \\ &= C_n^x \cdot \exp\left(x \ln \frac{p}{1-p} - (-n \ln(1-p))\right) \end{aligned}$$

$T_1(x) = x \Rightarrow$ Repartiția binomială aparține familiei exponențiale 1-dimensionale

b) $X \sim \text{Geom}(p)$

$$P(X=x) = p(1-p)^{x-1} \quad x=1,2,\dots$$

$$\begin{aligned} P(X=x) &= e^{\ln[p(1-p)^{x-1}]} = e^{\ln p + \ln(1-p)^{x-1}} = \\ &= e^{\ln p + (x-1)\ln(1-p)} = e^{\ln p + x\ln(1-p) - \ln(1-p)} = \\ &= e^{x\ln(1-p) + \ln \frac{p}{1-p}} = \exp(x\ln(1-p) - (-\ln(\frac{p}{1-p}))) \end{aligned}$$

$T_1(x) = x \Rightarrow$ Repartitie geometrică apartine familiei exponențiale 1-dimensionale

c) $X \sim \text{Pois}(\lambda)$

$$P(X=x) = e^{-\lambda} \cdot \frac{\lambda^x}{x!}, \quad x=0,1,\dots$$

$$\begin{aligned} P(X=x) &= \frac{1}{x!} \cdot e^{-\lambda} \cdot e^{x \ln \lambda} = \frac{1}{x!} \cdot e^{-\lambda + x \ln \lambda} = \\ &= \frac{1}{x!} \cdot \exp(x \ln \lambda - \lambda) \end{aligned}$$

$T_1(x) = x \Rightarrow$ Repartitie Poisson apartine familiei exponențiale 1-dimensionale

d) $X \sim \text{Exp}(\lambda)$

$$f(x) = \lambda e^{-\lambda x} \cdot \mathbb{1}_{\{x \geq 0\}}$$

$$f(x) = \mathbb{1}_{\{x \geq 0\}} \cdot e^{\ln \lambda - \lambda x} = \mathbb{1}_{\{x \geq 0\}} \exp(-\lambda x - (-\ln \lambda))$$

$= T_1(x) = x \Rightarrow$ Repartitie exponențială apartine familiei exponențiale 1-dimensionale

2) $X \sim \Gamma(\alpha, \beta)$ $\alpha > 0, \beta > 0$ α -shape, β scale

$$f(x) = \frac{1}{\Gamma(\alpha) \cdot \beta^\alpha} x^{\alpha-1} \cdot e^{-\frac{x}{\beta}} \cdot \mathbb{1}_{\{x>0\}}$$

$$f(x) = \mathbb{1}_{\{x>0\}} \cdot e^{-[\ln \Gamma(\alpha) + \ln \beta^\alpha]} \cdot e^{\ln x^{\alpha-1}} \cdot e^{-\frac{x}{\beta}}$$

$$= \mathbb{1}_{\{x>0\}} \cdot e^{-\ln \Gamma(\alpha) - \alpha \ln \beta + (\alpha-1) \ln x - \frac{x}{\beta}}$$

$$= \mathbb{1}_{\{x>0\}} \cdot \exp\left((\alpha-1) \ln x - \frac{x}{\beta} - \alpha \ln \beta - \ln \Gamma(\alpha)\right)$$

$T_1(x) = \ln x$, $T_2(x) = x \Rightarrow$ Repartitia Gamma apartine familiei exponentiale 2-dimensionale

4) $X \sim \text{Beta}(\alpha, \beta)$

$$f(x) = \frac{1}{B(\alpha, \beta)} x^{\alpha-1} (1-x)^{\beta-1} \cdot \mathbb{1}_{\{0<x<1\}}$$

$$f(x) = \mathbb{1}_{\{0<x<1\}} \cdot e^{-\ln B(\alpha, \beta)} \cdot e^{\ln x^{\alpha-1}} \cdot e^{\ln (1-x)^{\beta-1}}$$

$$= \mathbb{1}_{\{0<x<1\}} \cdot \exp\left((\alpha-1) \ln x + (\beta-1) \ln (1-x) - \ln B(\alpha, \beta)\right)$$

$T_1(x) = \ln x$, $T_2(x) = \ln(1-x) \Rightarrow$ Repartitia Beta apartine familiei exponentiale 2-dimensionale

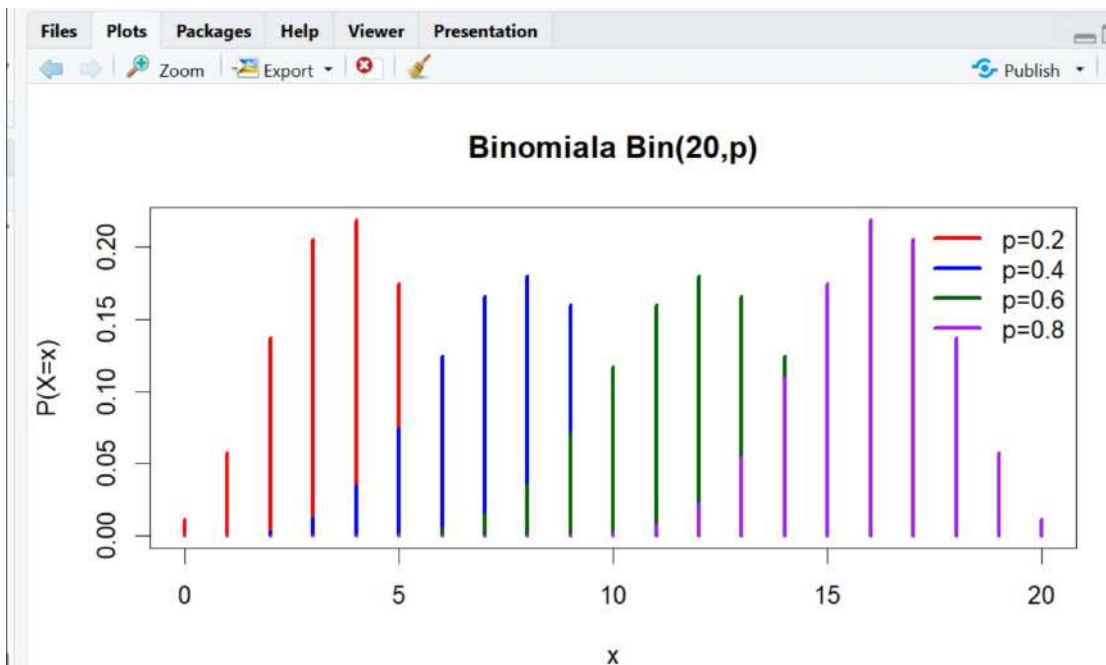
5) $X \sim \text{Unif}(a, b)$

Densitatea nu poate fi exprimată în forma canonică a familiei exponentiale, deoarece suportul distribuției depinde de parametri

```

305 #-----II. FAMILIE EXPONENTIALA-----
306 #-----1. APARTENENTA + REPREZENTARE GRAFICA-----
307 # culori folosite pentru cele 4 curbe/serii
308 col4 <- c("red", "blue", "darkgreen", "purple")
309
310
311 # =====
312 # 1) BINOMIALA Bin(20,p) (discreta)
313 # =====
314 n <- 20
315 x <- 0:n
316
317 y1 <- dbinom(x, n, 0.2)
318 y2 <- dbinom(x, n, 0.4)
319 y3 <- dbinom(x, n, 0.6)
320 y4 <- dbinom(x, n, 0.8)
321
322 plot(x, y1, type="h", lwd=3, col=col4[1],
323      main="Binomiala Bin(20,p)",
324      xlab="x", ylab="P(X=x)",
325      ylim=c(0, max(y1,y2,y3,y4)))
326
327 lines(x, y2, type="h", lwd=3, col=col4[2])
328 lines(x, y3, type="h", lwd=3, col=col4[3])
329 lines(x, y4, type="h", lwd=3, col=col4[4])
330
331 legend("topright",
332      legend=c("p=0.2", "p=0.4", "p=0.6", "p=0.8"),
333      col=col4, lwd=3, bty="n")
334
335

```



```

336 # =====
337 # 2) GEOMETRICA Geom(p) (R: x=0,1,2,...) (discreta)
338 # =====
339 x <- 0:10
340
341 y1 <- dgeom(x, 0.2)
342 y2 <- dgeom(x, 0.35)
343 y3 <- dgeom(x, 0.5)
344 y4 <- dgeom(x, 0.75)
345
346 plot(x, y1, type="h", lwd=3, col="red",
347      main="Geometrica Geom(p)",
348      xlab="x", ylab="P(X=x)",
349      ylim=c(0, max(y1,y2,y3,y4)))
350
351 lines(x, y2, type="h", lwd=3, col="blue")
352 lines(x, y3, type="h", lwd=3, col="darkgreen")
353 lines(x, y4, type="h", lwd=3, col="purple")
354
355 legend("topright",
356      legend=c("p=0.2", "p=0.35", "p=0.5", "p=0.75"),
357      col=c("red", "blue", "darkgreen", "purple"),
358      lwd=3)
359
360
361

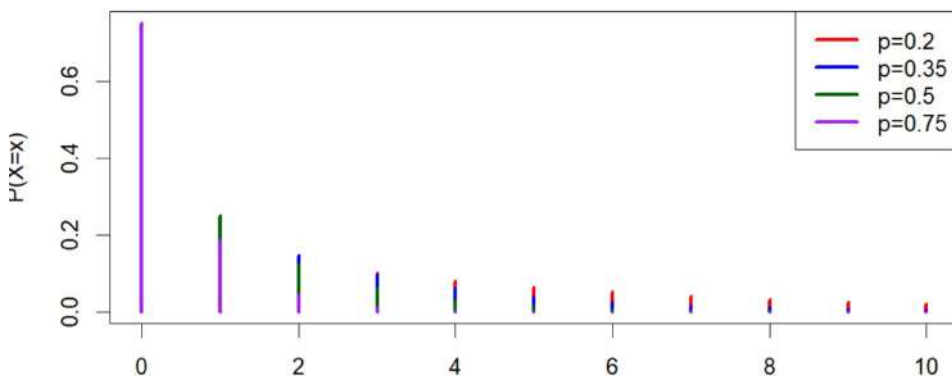
```

Fold code

Files Plots Packages Help Viewer Presentation

Zoom Export Publish

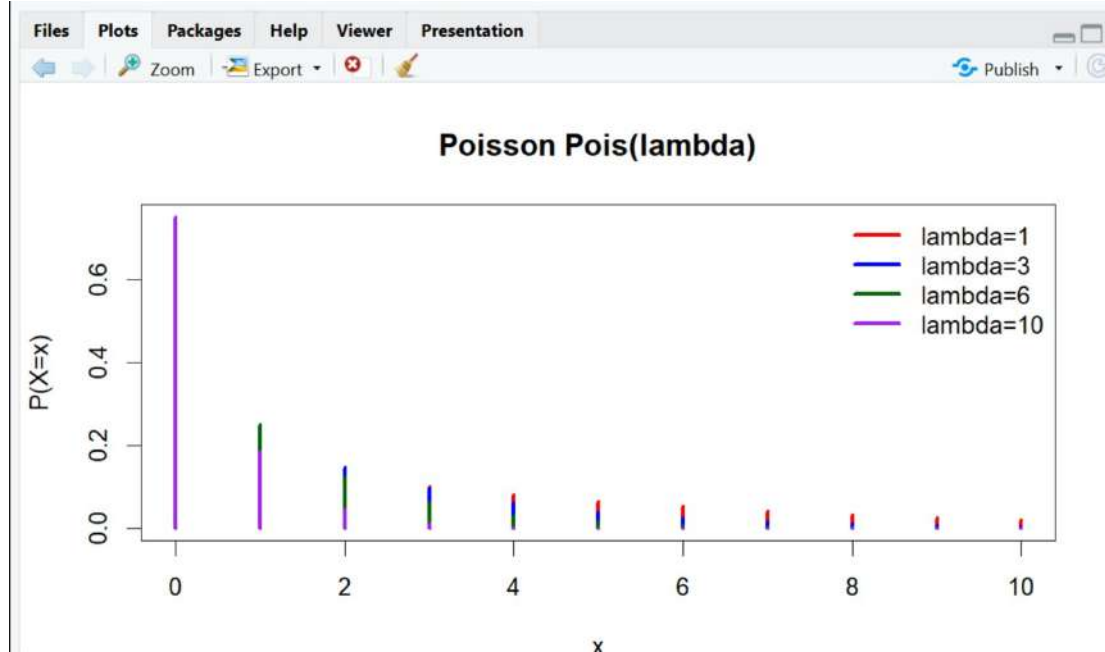
Geometrica Geom(p)



```

1 # =====
2 # 3) POISSON Pois(lambda) (discreta)
3 # =====
4 x <- 0:20
5
6 y1 <- dpois(x, 1)
7 y2 <- dpois(x, 3)
8 y3 <- dpois(x, 6)
9 y4 <- dpois(x, 10)
10
11 plot(x, y1, type="h", lwd=3, col=col4[1],
12      main="Poisson Pois(lambda)",
13      xlab="x", ylab="P(X=x)",
14      ylim=c(0, max(y1,y2,y3,y4)))
15
16 lines(x, y2, type="h", lwd=3, col=col4[2])
17 lines(x, y3, type="h", lwd=3, col=col4[3])
18 lines(x, y4, type="h", lwd=3, col=col4[4])
19
20 legend("topright",
21      legend=c("lambda=1", "lambda=3", "lambda=6", "lambda=10"),
22      col=col4, lwd=3, bty="n")
23

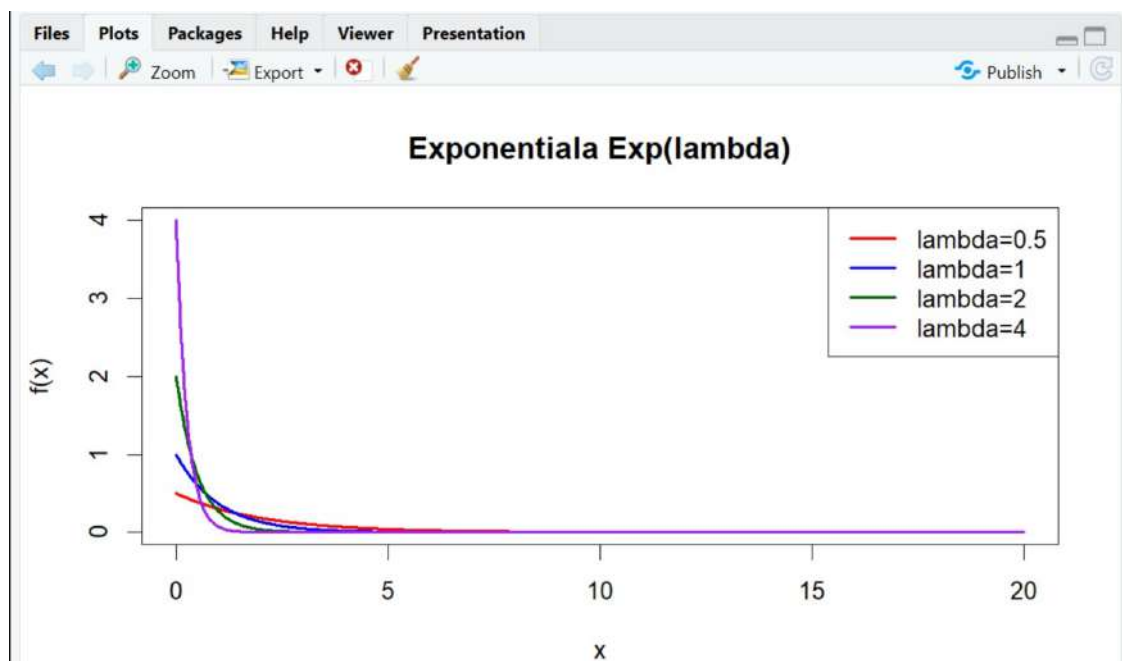
```

```

385 # =====
386 # 4) EXPONENTIALA Exp(lambda) (continua)
387 # =====
388 x <- seq(0, 20, 0.01)
389
390 y1 <- dexp(x, rate = 0.5)
391 y2 <- dexp(x, rate = 1)
392 y3 <- dexp(x, rate = 2)
393 y4 <- dexp(x, rate = 4)
394
395 plot(x, y1, type="l", lwd=2, col="red",
396      main="Exponentiala Exp(lambda)",
397      xlab="x", ylab="f(x)",
398      ylim=c(0, max(y1,y2,y3,y4)))
399
400 lines(x, y2, lwd=2, col="blue")
401 lines(x, y3, lwd=2, col="darkgreen")
402 lines(x, y4, lwd=2, col="purple")
403
404 legend("topright",
405      legend=c("lambda=0.5", "lambda=1", "lambda=2", "lambda=4"),
406      col=c("red", "blue", "darkgreen", "purple"),
407      lwd=2)
408

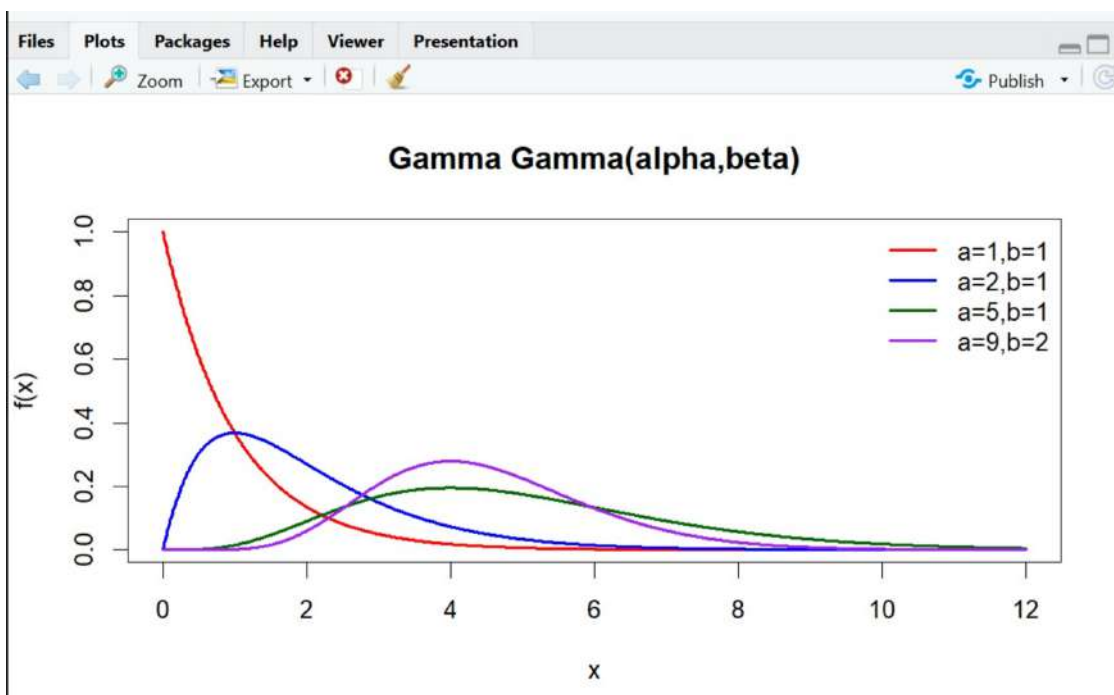
```



```

410 # =====
411 # 5) GAMMA Gamma(alpha,beta) (continua)
412 # In R: shape=alpha, rate=beta
413 # =====
414 x <- seq(0, 12, 0.01)
415
416 y1 <- dgamma(x, shape=1, rate=1)
417 y2 <- dgamma(x, shape=2, rate=1)
418 y3 <- dgamma(x, shape=5, rate=1)
419 y4 <- dgamma(x, shape=9, rate=2)
420
421 plot(x, y1, type="l", lwd=2, col=col4[1],
422      main="Gamma Gamma(alpha,beta)",
423      xlab="x", ylab="f(x)",
424      ylim=c(0, max(y1,y2,y3,y4)))
425
426 lines(x, y2, lwd=2, col=col4[2])
427 lines(x, y3, lwd=2, col=col4[3])
428 lines(x, y4, lwd=2, col=col4[4])
429
430 legend("topright",
431      legend=c("a=1,b=1","a=2,b=1","a=5,b=1","a=9,b=2"),
432      col=col4, lwd=2, bty="n")
433
434

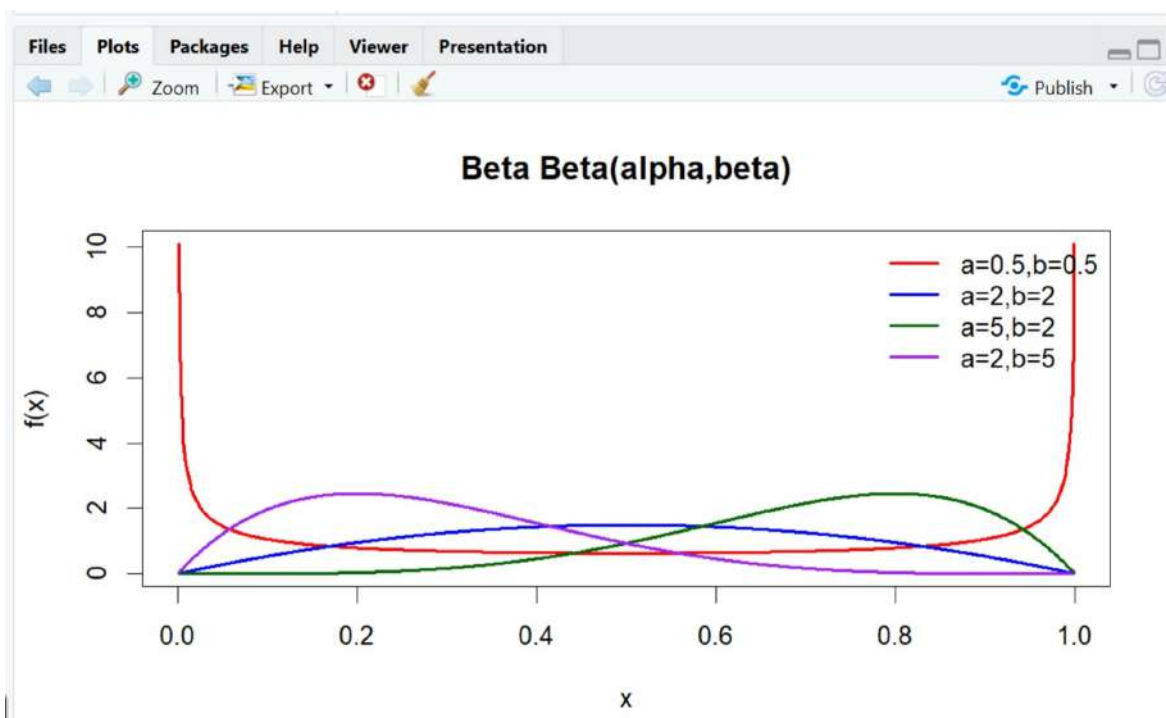
```



```

36 # 6) BETA Beta(alpha,beta) (continua, pe (0,1))
37 # =====
38 x <- seq(0.001, 0.999, 0.001)
39
40 y1 <- dbeta(x, 0.5, 0.5)
41 y2 <- dbeta(x, 2, 2)
42 y3 <- dbeta(x, 5, 2)
43 y4 <- dbeta(x, 2, 5)
44
45 plot(x, y1, type="l", lwd=2, col=col4[1],
46      main="Beta Beta(alpha,beta)",
47      xlab="x", ylab="f(x)",
48      ylim=c(0, max(y1,y2,y3,y4)))
49
50 lines(x, y2, lwd=2, col=col4[2])
51 lines(x, y3, lwd=2, col=col4[3])
52 lines(x, y4, lwd=2, col=col4[4])
53
54 legend("topright",
55      legend=c("a=0.5,b=0.5", "a=2,b=2", "a=5,b=2", "a=2,b=5"),
56      col=col4, lwd=2, bty="n")
57

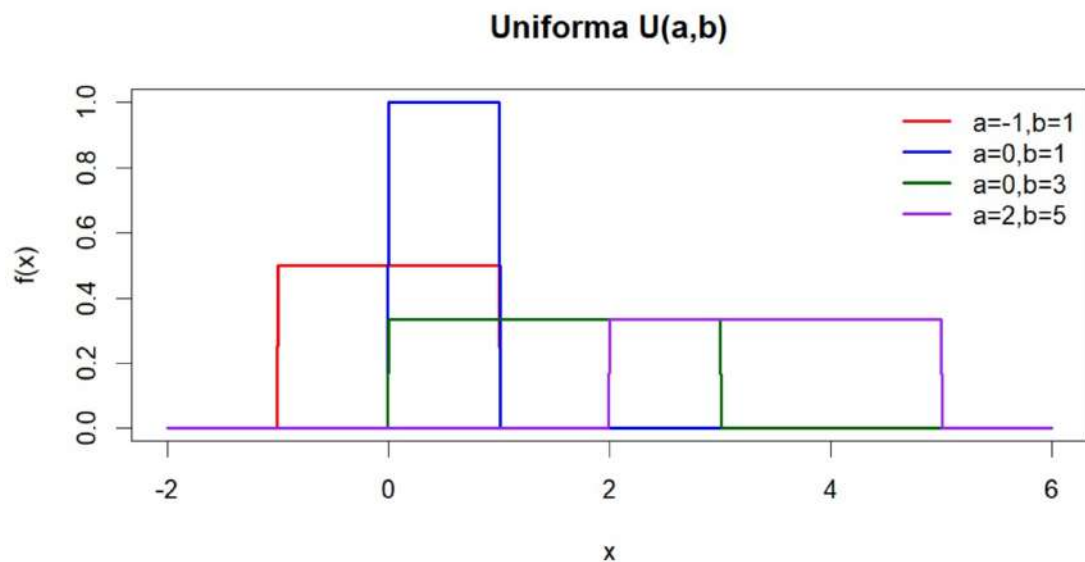
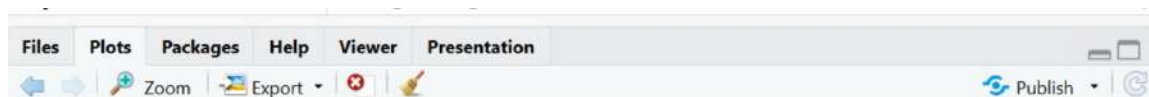
```




```

459 # =====
460 # 7) UNIFORMA U(a,b) (continua)
461 # =====
462 x <- seq(-2, 6, 0.01)
463
464 y1 <- dunif(x, min=-1, max=1)
465 y2 <- dunif(x, min=0, max=1)
466 y3 <- dunif(x, min=0, max=3)
467 y4 <- dunif(x, min=2, max=5)
468
469 plot(x, y1, type="l", lwd=2, col=col4[1],
470      main="Uniforma U(a,b)",
471      xlab="x", ylab="f(x)",
472      ylim=c(0, max(y1,y2,y3,y4)))
473
474 lines(x, y2, lwd=2, col=col4[2])
475 lines(x, y3, lwd=2, col=col4[3])
476 lines(x, y4, lwd=2, col=col4[4])
477
478 legend("topright",
479      legend=c("a=-1,b=1", "a=0,b=1", "a=0,b=3", "a=2,b=5"),
480      col=col4, lwd=2, bty="n")
481
482

```



2. Concavitătea funcției de log-verosimilitate

- a) Construiți funcția de log-verosimilitate pentru familia exponențială și demonstrați că aceasta este concavă (construiți matricea hessiană și arătați că este negativă sau semi-negativ definită)
- b) Particularizați forma funcției de log-verosimilitate pentru repartițiile de la punctul 1 care fac parte din familia exponențială cu un parametru sau cu doi parametri.
- c) Pentru repartițiile cu un parametru reprezentați grafic funcția de log-verosimilitate (în R) și determinați punctul de maxim folosind funcția optimize.
- d) Pentru repartițiile cu doi parametri fixați, pe rând, unul dintre parametri (alegeți o valoare particulară), apoi reprezentați grafic log-verosimilitatea în funcție de celălalt parametru și determinați punctul de maxim folosind funcția optimize.

5) Concavitătea funcției de log-verosimilitate:

$$1) f_X(x, \theta) = h(x) \cdot \exp\left(\sum_{i=1}^n \eta_i(\theta) \cdot T_i(x) - A(\theta)\right)$$

• Construcția log-verosimilității:

Pentru o singură observație x , funcția de log-verosimilitate este dată de:

$$l(\tilde{\theta}, x) = \ln h(x) + \sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x) - \tilde{A}(\tilde{\theta}), \text{ unde } \tilde{\theta}_i \text{ reprezintă parametrii naturali } \eta_i(\theta).$$

• Demonstrarea concavității prin matricea Hessiană:

Arătăm că $H = \left(\frac{\partial^2 l(\tilde{\theta}, x)}{\partial \tilde{\theta}_i \partial \tilde{\theta}_j} \right)$ este negativ definită.

$$\frac{\partial l}{\partial \tilde{\theta}_j} = T_j(x) - \frac{\partial \tilde{A}(\tilde{\theta})}{\partial \tilde{\theta}_j}$$

$$\frac{\partial^2 l}{\partial \tilde{\theta}_i \partial \tilde{\theta}_j} = - \frac{\partial^2 \tilde{A}(\tilde{\theta})}{\partial \tilde{\theta}_i \partial \tilde{\theta}_j}$$

Revenim la constanta de normalizare $\tilde{A}(\tilde{\theta})$ care este dată de: $\tilde{A}(\tilde{\theta}) = \ln \left(\int h(x) \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx \right)$
 $\stackrel{\text{not}}{=} \ln B(\tilde{\theta})$

$$\frac{\partial \tilde{A}(\tilde{\theta})}{\partial \tilde{\theta}_j} = \frac{\partial}{\partial \tilde{\theta}_j} \ln B(\tilde{\theta}) = \frac{1}{B(\tilde{\theta})} \cdot \left(\frac{\partial}{\partial \tilde{\theta}_j} B(\tilde{\theta}) \right) =$$

$$= \frac{1}{B(\tilde{\theta})} \int h(x) \cdot \frac{\partial}{\partial \tilde{\theta}_j} \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx =$$

$$= \frac{1}{B(\tilde{\theta})} \int h(x) \cdot T_j(x) \cdot \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx = E[T_j(x)]$$

$$\begin{aligned}
 \frac{\partial^2 A(\tilde{\theta})}{\partial \tilde{\theta}_i \partial \tilde{\theta}_j} &= \frac{\partial}{\partial \tilde{\theta}_i} \left(\frac{\int h(x) T_j(x) \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx}{\int h(x) \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx} \right) = \\
 &= \frac{\int h(x) T_i(x) T_j(x) e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx \int h(x) e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx -}{\left(\int h(x) \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx \right)^2} \\
 &= \frac{\int h(x) T_j(x) \cdot e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx \cdot \int h(x) \cdot T_i(x) \cdot e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx}{\left(\int h(x) \exp\left(\sum_{i=1}^n \tilde{\theta}_i \cdot T_i(x)\right) dx \right)^2} = \\
 &= \frac{1}{B(\tilde{\theta})} \int h(x) T_i(x) T_j(x) e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx - \frac{1}{B(\tilde{\theta})} \cdot \int h(x) T_j(x) \cdot e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx \cdot \\
 &\quad \cdot \frac{1}{B(\tilde{\theta})} \cdot \int h(x) \cdot T_i(x) \cdot e^{\sum_{i=1}^n \tilde{\theta}_i T_i(x)} dx.
 \end{aligned}$$

$$\begin{aligned}
 \text{Deci, } \frac{\partial^2 A(\tilde{\theta})}{\partial \tilde{\theta}_i \partial \tilde{\theta}_j} &= \mathbb{E}[T_i(x) T_j(x)] - \mathbb{E}[T_i(x)] \cdot \mathbb{E}[T_j(x)] \\
 &= \text{Cov}(T_i(x), T_j(x)).
 \end{aligned}$$

$$\begin{aligned}
 \text{Am obținut că } H &= -\text{Cov}(T_i(x), T_j(x)) \\
 H &= -\text{Cov}(T(x))
 \end{aligned}$$

Rămâne de arătat că pentru un vector aleator n dimensional x , matricea $\text{Cov}(x)$ este semi pozitiv-definită:

Fie $y \in \mathbb{R}^n$ și $y := y^t (x - \mathbb{E}[x])$, atunci:

$$\begin{aligned}
 0 \leq \mathbb{E}[y^2] &= \mathbb{E}[y \cdot y^t] = y^t \mathbb{E}[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^t] y \\
 &= y^t \text{Cov}(x) y \quad \checkmark \Rightarrow
 \end{aligned}$$

$\Rightarrow H$ semi-negativ definită $\Rightarrow \ell(\tilde{\theta}, x)$ concavă

b) Fie $x_1, x_2, \dots, x_m$ iid cu valoare de selecție $x_1, \dots, x_m$.

i) $X \sim \text{Bin}(m, p) \stackrel{(1)}{\Rightarrow} f(x, p) = C_m^x \cdot \exp(x \cdot \ln \frac{p}{1-p} - (-m \ln(1-p))) =$

$\Rightarrow$ • parametrul natural $\eta = \ln \frac{p}{1-p}$

• statistica suficientă $T(X) = X$

• constanta de log-normalizare $A = -m \ln(1-p)$.

Exprimăm în funcție de η .

$$\ln \frac{p}{1-p} = \eta \Rightarrow \frac{p}{1-p} = e^\eta \Rightarrow p = e^\eta - p e^\eta \Rightarrow$$

$$\Rightarrow p = \frac{e^\eta}{1+e^\eta} \Rightarrow A(\eta) = -m \ln \frac{1}{1+e^\eta} = m \ln(1+e^\eta)$$

$$L(\eta, x) = \prod_{i=1}^m f(x_i) \Rightarrow \ell(\eta) = \underbrace{\sum_{i=1}^m C_m^{x_i}}_{\text{constantă}} + \eta \sum_{i=1}^m x_i - m \ln(1+e^\eta)$$

ii) $X \sim \text{Geom}(p) \stackrel{(1)}{\Rightarrow} f(x, p) = \exp(x \ln(1-p) + \ln \frac{p}{1-p}) =$

$\Rightarrow$ • parametrul natural $\eta = \ln(1-p) \Rightarrow p = 1 - e^\eta$

• statistica suficientă $T(X) = X$

• constanta de log-normalizare $A(\eta) = -\ln \frac{p}{1-p} = -\ln \frac{1-e^\eta}{e^\eta} =$

$\Rightarrow A(\eta) = \eta - \ln(1-e^\eta) \Rightarrow f(x) = \exp(x \cdot \eta - A(\eta))$

$$\ell(\eta) = \sum_{i=1}^m x_i \cdot \eta - (m \cdot \ln(1-e^\eta)) \cdot m$$

$$= \eta \left(\sum_{i=1}^m x_i - m \right) + m \cdot \ln(1-e^\eta)$$

iii) $X \sim \text{Pois}(\lambda) \stackrel{(1)}{\Rightarrow} f(x) = \frac{1}{x!} \cdot \exp(x \ln \lambda - \lambda) =$

$\Rightarrow \eta = \ln \lambda, T(X) = X, A(\eta) = \lambda = e^\eta$

$$f(x) = \frac{1}{x!} \cdot \exp(x \cdot \eta - e^\eta)$$

$$\ell(\eta) = \underbrace{\sum_{i=1}^m \ln \left(\frac{1}{x_i!} \right)}_{\text{constantă}} + \eta \cdot \sum_{i=1}^m x_i - m \cdot e^\eta$$

$$iv) X \sim \text{Exp}(\lambda) \stackrel{(4)}{\Rightarrow} f(x) = \begin{cases} \exp(-\lambda x - (-\ln \lambda)) & x \geq 0 \\ 0 & x < 0 \end{cases} \Rightarrow \begin{cases} \eta = -\lambda \\ T(x) = x \\ A(\eta) = -\ln \lambda \end{cases}$$

$$A(\eta) = -\ln \lambda = -\ln(-\eta).$$

$$\text{Deci, } f(x) = \begin{cases} \exp(\eta \cdot x - (-\ln(-\eta))) & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$l(\eta) = \sum_{i=1}^m (x_i \cdot \eta) + m \log(-\eta)$$

$$l(\eta) = \eta \sum_{i=1}^m x_i + m \ln(-\eta).$$

$$v) X \sim \text{Gamma}(\alpha, \beta), \alpha > 0, \beta > 0 \stackrel{(4)}{\Rightarrow} f(x) = \exp((\alpha-1) \ln x - \frac{x}{\beta} - \alpha \ln \beta - \ln \Gamma(\alpha)) \cdot \begin{cases} x > 0 \\ 0 & x \leq 0 \end{cases}$$

$$\Rightarrow \text{Parametri naturali: } \theta_1 = \alpha - 1 \text{ și } \theta_2 = -\frac{1}{\beta}$$

$$\bullet \text{ Statisticile suficiente: } T_1(x) = \ln x \text{ și } T_2(x) = x.$$

$$\bullet A(\alpha, \beta) = \alpha \ln \beta + \ln \Gamma(\alpha)$$

$$l(\alpha, \beta, x) = \sum_{j=1}^m [(\alpha-1) \ln x_j - \frac{x_j}{\beta}] - m(\alpha \ln \beta + \ln \Gamma(\alpha)) =$$

$$= (\alpha-1) \sum_{j=1}^m \ln x_j - \frac{1}{\beta} \sum_{j=1}^m x_j - m \alpha \ln \beta - m \ln \Gamma(\alpha).$$

$$vi) X \sim \text{Beta}(\alpha, \beta) \stackrel{(5)}{\Rightarrow} f(x) = \begin{cases} \exp((\alpha-1) \ln x + (\beta-1) \ln(1-x) - \ln B(\alpha, \beta)) & 0 < x < 1 \\ 0 & \text{altfel} \end{cases} \Rightarrow$$

$$\Rightarrow \begin{cases} \theta_1 = \alpha - 1, \theta_2 = \beta - 1 \\ T_1(x) = \ln x, T_2(x) = \ln(1-x) \\ A(\alpha, \beta) = \ln B(\alpha, \beta) \end{cases}$$

$$l(\alpha, \beta, x) = \sum_{j=1}^m \ln f(x_j) = \sum_{j=1}^m (\alpha-1) \ln x_j + (\beta-1) \sum_{j=1}^m \ln(1-x_j) - m \ln B(\alpha, \beta).$$

c)

```

484 # -----geometrica-----
485 # datele observate
486 m <- 50
487 date_geom <- rgeom(m, prob = 0.25) # exemplu de date generate
488
489 #Definirea functiei de log-verosimilitate (l)
490 # Calculele se bazeaza pe forma: l = ln(1-p)*sum(x) + m*ln(p)
491 log_lik_manual <- function(p, x) {
492   # protejam logaritmul pentru a evita valori negative sau zero
493   if (p <= 0 | p >= 1) return(-Inf)
494
495   m <- length(x)
496   suma_x <- sum(x)
497
498   # Implementarea formulei deduse anterior
499   valoare <- log(1 - p) * suma_x + m * log(p)
500   return(valoare)
501 }
502
503 # Gasim punctul de maxim folosind functia optimize
504 # Cautam in intervalul (0, 1) deoarece p este o probabilitate
505 # Conform calculelor de concavitate, avem un singur maxim global
506 rezultat <- optimize(f = log_lik_manual, interval = c(0, 0.99),
507                      x = date_geom, maximum = TRUE)
508
509 # Afisarea rezultatului in consola
510 cat("Punctul de maxim (MLE) pentru p este:", rezultat$maximum)
511
512 # Reprezentare grafica pentru vizualizarea concavitatii
513 p_grid <- seq(0.01, 0.9, length.out = 100)
514 y_plot <- sapply(p_grid, log_lik_manual, x = date_geom)
515
516 plot(p_grid, y_plot, type = "l", col = "blue", lwd = 2,
517      main = "Graficul functiei de log-verosimilitate (Geometrica)",
518      xlab = "Parametrul p", ylab = "Log-Likelihood")
519
520 # Marcam punctul de maxim determinat prin optimize
521 abline(v = rezultat$maximum, col = "red", lty = 2)
522

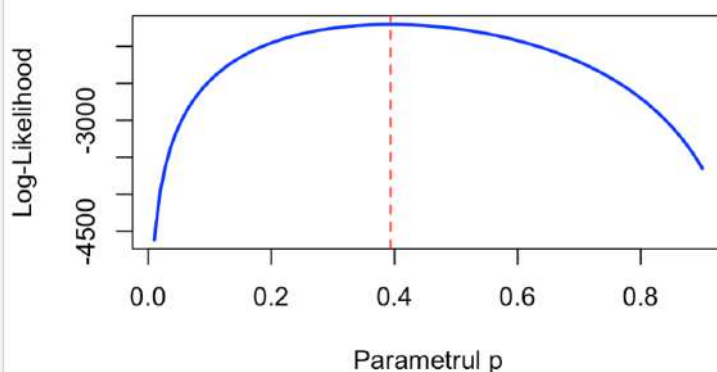
```

```

> cat("Punctul de maxim (MLE) pentru p este:", rezultat$maximum)
Punctul de maxim (MLE) pentru p este: 0.2222089>

```

Graficul functiei de log-verosimilitate (Geometrica)



```

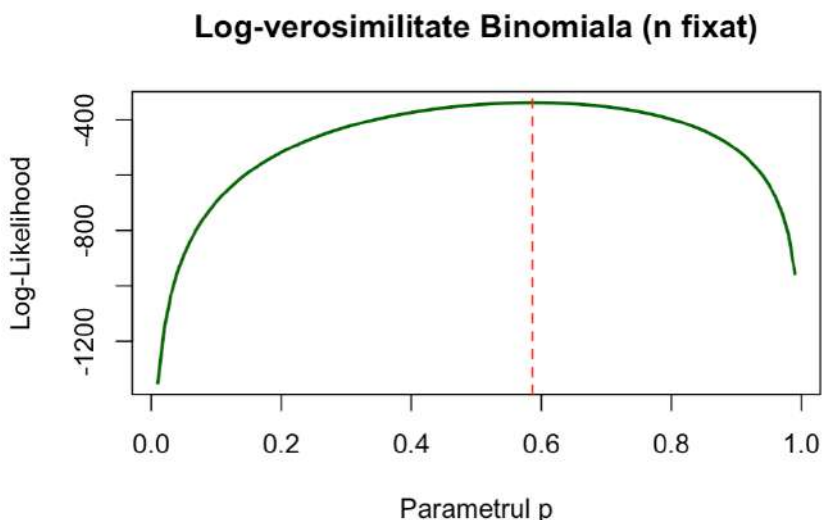
524 #-----binomiala-----
525 # datele observate
526 m <- 50          # numarul de observatii din esantion
527 n_fixat <- 10     # numarul de incercari pentru fiecare observatie
528 date_bin <- rbinom(m, size = n_fixat, prob = 0.6) # exemplu de date generate
529
530 # Definirea functiei de log-verosimilitate
531 log_lik_bin_manual <- function(p, x, n) {
532   # protejam logaritmul
533   if (p <= 0 | p >= 1) return(-Inf)
534
535   m <- length(x)
536   suma_x <- sum(x)
537
538   # Formula bazata pe descompunerea familiei exponentiale
539   l = suma_x * log(p) + (m*n - suma_x) * log(1-p)
540   valoare <- suma_x * log(p) + (m * n - suma_x) * log(1 - p)
541   return(valoare)
542 }
543
544 # Gasim punctul de maxim folosind optimize
545 # Cautam in intervalul (0, 1) pentru probabilitatea p
546 rezultat_bin <- optimize(f = log_lik_bin_manual, interval = c(0.01, 0.99),
547                           x = date_bin, n = n_fixat, maximum = TRUE)
548
549 cat("Punctul de maxim (MLE) pentru p la Binomiala este:", rezultat_bin$maximum)
550
551 # Reprezentare grafica
552 p_grid <- seq(0.01, 0.99, length.out = 100)
553 y_plot <- sapply(p_grid, log_lik_bin_manual, x = date_bin, n = n_fixat)
554
555 plot(p_grid, y_plot, type = "l", col = "darkgreen", lwd = 2,
556      main = "Log-verosimilitate Binomiala (n fixat)",
557      xlab = "Parametrul p", ylab = "Log-Likelihood")
558
559 # Marcam punctul de maxim determinat
560 abline(v = rezultat_bin$maximum, col = "red", lty = 2)
561

```

```

> cat("Punctul de maxim (MLE) pentru p la Binomiala este: ", rezu
Punctul de maxim (MLE) pentru p la Binomiala este: 0.5859971>
# Reprezentare grafica

```



```

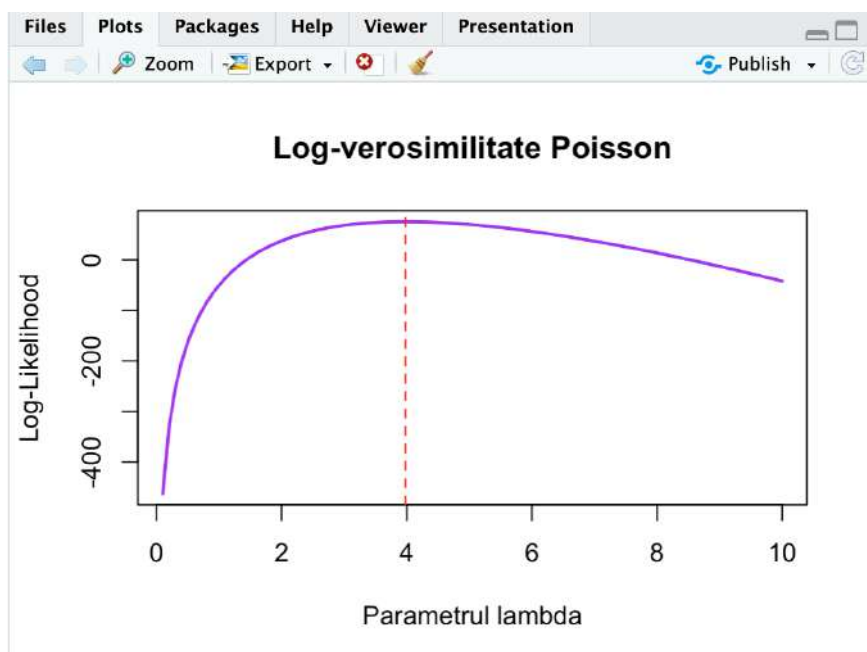
563 #| Poisson
564 # datele observate
565 m <- 50
566 lambda_real <- 4
567 date_pois <- rpois(m, lambda = lambda_real) # exemplu de date generate
568
569 # Definirea functiei de log-verosimilitate
570 log_lik_pois_manual <- function(lambda, x) {
571   # lambda trebuie sa fie strict pozitiv
572   if (lambda <= 0) return(-Inf)
573
574   m <- length(x)
575   suma_x <- sum(x)
576
577   # Formula: ln(lambda) * sum(x) - m * lambda
578   valoare <- log(lambda) * suma_x - m * lambda
579   return(valoare)
580 }
581
582 # Gasim punctul de maxim folosind optimize
583 # Cautam intr-un interval rezonabil pentru lambda (ex: 0.1 la 20)
584 rezultat_pois <- optimize(f = log_lik_pois_manual, interval = c(0.01, 20),
585                           x = date_pois, maximum = TRUE)
586
587 cat("Punctul de maxim (MLE) pentru lambda la Poisson este:", rezultat_pois$maximum)
588
589 # Reprezentare grafica
590 lambda_grid <- seq(0.1, 10, length.out = 100)
591 y_plot <- sapply(lambda_grid, log_lik_pois_manual, x = date_pois)
592
593 plot(lambda_grid, y_plot, type = "l", col = "purple", lwd = 2,
594       main = "Log-verosimilitate Poisson",
595       xlab = "Parametrul lambda", ylab = "Log-Likelihood")
596
597 # Marcam punctul de maxim determinat
598 abline(v = rezultat_pois$maximum, col = "red", lty = 2)
599

```

```

> cat("Punctul de maxim (MLE) pentru lambda la Poisson este:", rezul
Punctul de maxim (MLE) pentru lambda la Poisson este: 3.939999>
> # Reprezentare grafica

```

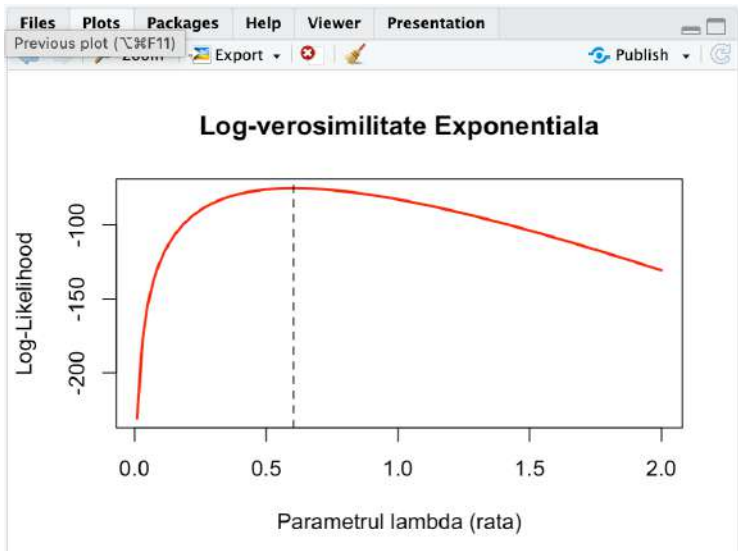



```

601 # Exponentiala
602 # datele observate
603 m <- 50
604 lambda_real <- 0.5
605 set.seed(101)
606 date_exp <- rexp(m, rate = lambda_real) # exemplu de date generate
607
608 # Definirea functiei de log-verosimilitate
609 log_lik_exp_manual <- function(lambda, x) {
610   # lambda (rata) trebuie sa fie strict pozitiva
611   if (lambda <= 0) return(-Inf)
612
613   m <- length(x)
614   suma_x <- sum(x)
615
616   # Formula dedusa: m * ln(lambda) - lambda * sum(x)
617   valoare <- m * log(lambda) - lambda * suma_x
618   return(valoare)
619 }
620
621 # Gasim punctul de maxim folosind functia optimize
622 # Pentru exponentiala, MLE-ul teoretic este 1/media(x)
623 rezultat_exp <- optimize(f = log_lik_exp_manual, interval = c(0.01, 10),
624   x = date_exp, maximum = TRUE)
625
626 cat("Punctul de maxim (MLE) pentru lambda la Exponentiala este:", rezultat_exp$maximum)
627
628 # Reprezentare grafica
629 lambda_grid <- seq(0.01, 2, length.out = 100)
630 y_plot <- sapply(lambda_grid, log_lik_exp_manual, x = date_exp)
631
632 plot(lambda_grid, y_plot, type = "l", col = "red", lwd = 2,
633   main = "Log-verosimilitate Exponentiala",
634   xlab = "Parametrul lambda (rata)", ylab = "Log-Likelihood")
635
636 # Marcam punctul de maxim determinat prin optimize
637 abline(v = rezultat_exp$maximum, col = "black", lty = 2)
638

```

> cat("Punctul de maxim (MLE) pentru lambda la Exponentiala este:", rezultat_exp\$maximum)
Punctul de maxim (MLE) pentru lambda la Exponentiala este: 0.6043556>



d)

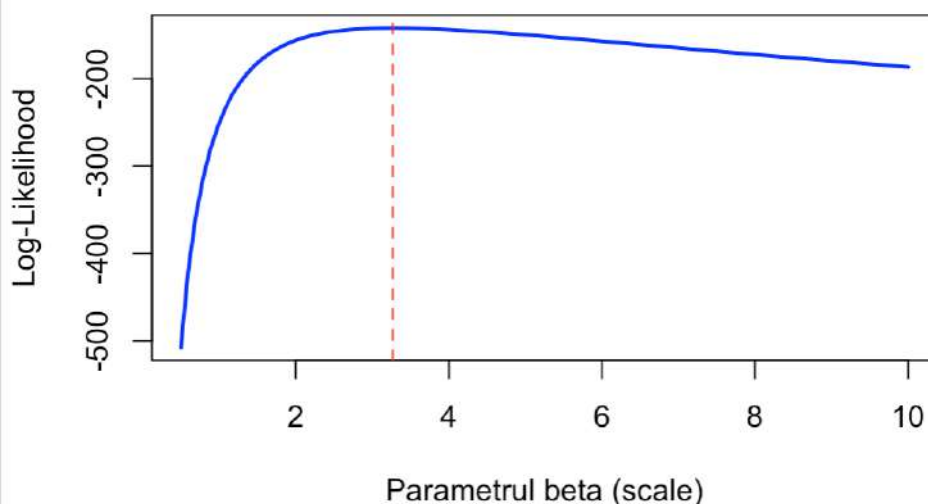
```

641 #Gamma (fixam alpha, optimizam beta)
642 alpha_fixat <- 2
643 beta_real <- 3
644 date_gamma <- rgamma(50, shape = alpha_fixat, scale = beta_real)
645
646 # Functia de log-verosimilitate pentru beta (cu alpha fixat)
647 # Formula: (alpha-1)*sum(log(x)) - (1/beta)*sum(x) - m*alpha*log(beta) - m*log(gamma(alpha))
648 log_lik_gamma_beta <- function(beta, x, alpha) {
649   if (beta <= 0) return(-Inf)
650   m <- length(x)
651   valoare <- (alpha - 1) * sum(log(x)) - (1/beta) * sum(x) -
652     m * alpha * log(beta) - m * lgamma(alpha)
653   return(valoare)
654 }
655
656 # Optimizare pentru beta
657 rezultat_gamma <- optimize(f = log_lik_gamma_beta, interval = c(0.1, 20),
658   x = date_gamma, alpha = alpha_fixat, maximum = TRUE)
659
660 cat("Pentru Gamma (alpha=2), maximul pentru beta este:", rezultat_gamma$maximum)
661
662 # Grafic
663 beta_grid <- seq(0.5, 10, length.out = 100)
664 y_gamma <- sapply(beta_grid, log_lik_gamma_beta, x = date_gamma, alpha = alpha_fixat)
665 plot(beta_grid, y_gamma, type = "l", col = "blue", lwd = 2,
666   main = "Log-verosimilitate Gamma (beta) cu alpha=2 fixat",
667   xlab = "Parametrul beta (scale)", ylab = "Log-Likelihood")
668 abline(v = rezultat_gamma$maximum, col = "red", lty = 2)
669
670

```

Pentru Gamma (alpha=2), maximul pentru beta este: 3.264849>

Log-verosimilitate Gamma (beta) cu alpha=2 fixat

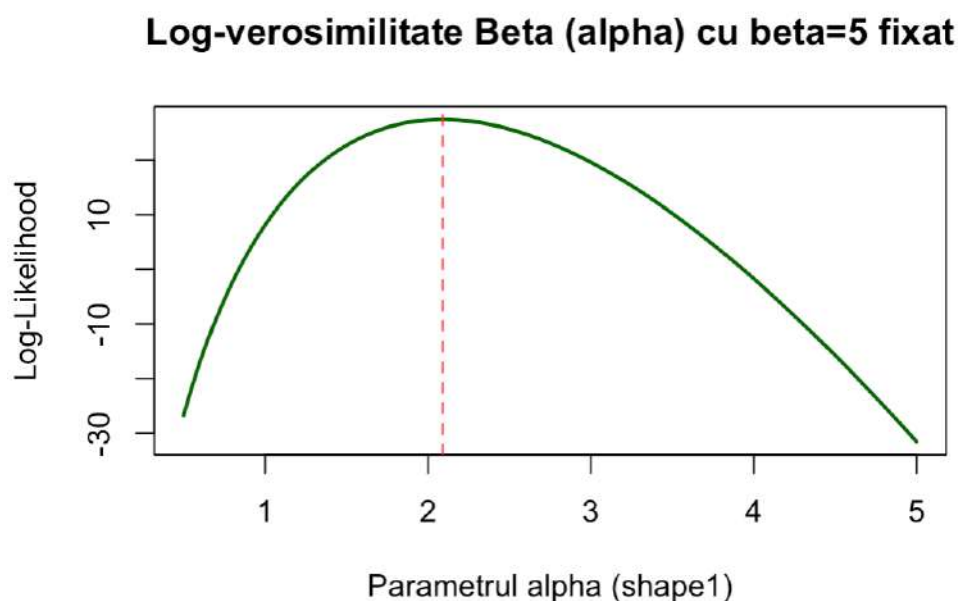


```

671 #beta (fixam beta, optimizam alpha)
672 set.seed(456)
673 alpha_real <- 2
674 beta_fixat <- 5
675 date_beta <- rbeta(50, shape1 = alpha_real, shape2 = beta_fixat)
676
677 # Functia de log-verosimilitate pentru alpha (cu beta fixat)
678 # Formula: (alpha-1)*sum(log(x)) + (beta-1)*sum(log(1-x)) - m*log(beta_func(alpha, beta))
679 log_lik_beta_alpha <- function(alpha, x, beta) {
680   if (alpha <= 0) return(-Inf)
681   m <- length(x)
682   # log(beta_func) se scrie in R ca lbeta(alpha, beta)
683   valoare <- (alpha - 1) * sum(log(x)) + (beta - 1) * sum(log(1 - x)) -
684     m * lbeta(alpha, beta)
685   return(valoare)
686 }
687
688 # Optimizare pentru alpha
689 rezultat_beta <- optimize(f = log_lik_beta_alpha, interval = c(0.1, 10),
690   x = date_beta, beta = beta_fixat, maximum = TRUE)
691
692 cat("Pentru Beta (beta=5), maximul pentru alpha este:", rezultat_beta$maximum)
693
694 # Grafic
695 alpha_grid <- seq(0.5, 5, length.out = 100)
696 y_beta <- sapply(alpha_grid, log_lik_beta_alpha, x = date_beta, beta = beta_fixat)
697 plot(alpha_grid, y_beta, type = "l", col = "darkgreen", lwd = 2,
698   main = "Log-verosimilitate Beta (alpha) cu beta=5 fixat",
699   xlab = "Parametrul alpha (shape1)", ylab = "Log-Likelihood")
700 abline(v = rezultat_beta$maximum, col = "red", lty = 2)
701
702

```

Pentru Beta (beta=5), maximul pentru alpha este: 2.089378



3. Estimatori: metoda verosimilității maxime și metoda momentelor

a) Pentru familiile exponențiale cu un parametru și respectiv cu doi parametri calculați estimatorul de verosimilitate maximă și estimatorul obținut prin metoda momentelor.

b) Ilustrați în R, pentru un eșantion de dimensiune $n = 1000$ generat de voi, faptul că estimațiile obținute prin cele două metode sunt foarte apropiate de valoarea adevărată a parametrului de interes, pentru toate repartițiile de la punctul 1 care fac parte din familia exponențială.

a) i) $X \sim \text{Bin}(m, p)$

$$l(p) = \ln\left(\frac{p}{1-p}\right) \cdot \sum_{j=1}^m x_j + m \ln(1-p) + \text{constante}$$

$$\frac{\partial l}{\partial p} = \frac{1-p}{p} \cdot \frac{1-p+x}{(1-p)^2} \cdot m \cdot \bar{x} + m \ln \frac{1}{1-p} = \frac{m\bar{x} - mmp}{p(1-p)}$$

$$\frac{\partial l}{\partial p} = 0 \Leftrightarrow m\bar{x} = mmp \Leftrightarrow \hat{p}_{MLE} = \frac{\bar{x}}{m}$$

$$E[X] = mp = \bar{x} \Rightarrow \hat{p}_{MM} = \frac{\bar{x}}{m}$$

$$\text{Deci, } \hat{p}_{MLE} = \hat{p}_{MM} = \frac{\bar{x}}{m}$$

ii) $X \sim \text{Pois}(\lambda)$

$$l(\lambda) = \sum_{i=1}^m x_i \ln \lambda - m\lambda$$

$$\frac{\partial l}{\partial \lambda} = \frac{1}{\lambda} \cdot m \cdot \bar{x} - m = \frac{m(\bar{x} - \lambda)}{\lambda}$$

$$\frac{\partial l}{\partial \lambda} = 0 \Leftrightarrow \bar{x} = \lambda \Rightarrow \hat{\lambda}_{MLE} = \bar{x} \quad \left. \vphantom{\frac{\partial l}{\partial \lambda} = 0} \right\} \Rightarrow \hat{\lambda}_{MLE} = \hat{\lambda}_{MM} = \bar{x}$$

$$E[X] = \lambda = \bar{x} \Rightarrow \hat{\lambda}_{MM} = \bar{x}$$

iii) $X \sim \text{Geom}(p)$

$$l(p) = \ln(1-p) \sum x_i + m \ln p$$

$$\frac{dl}{dp} = \frac{-1}{1-p} \cdot m \cdot \bar{x} + \frac{m}{p} = \frac{m(-p\bar{x} + 1-p)}{p(1-p)}$$

$$\frac{dl}{dp} = 0 \Leftrightarrow p(\bar{x} + 1) = 1 \Rightarrow \hat{p}_{MLE} = \frac{1}{\bar{x} + 1}$$

$$E[X] = \frac{1-p}{p} = \bar{x} \Rightarrow \bar{x} \cdot p = 1-p$$

$$p(1 + \bar{x}) = 1 \Rightarrow \hat{p}_{MM} = \frac{1}{1 + \bar{x}}$$

$$\text{Deci, } \hat{p}_{MLE} = \hat{p}_{MM}$$

iv) $X \sim \text{Exp}(\lambda)$

$$l(\lambda) = m \ln \lambda - \lambda \cdot \sum_{j=1}^m x_j$$

$$\frac{\partial l}{\partial \lambda} = \frac{m}{\lambda} - m \cdot \bar{x} = \frac{m(1 - \lambda \cdot \bar{x})}{\lambda}$$

$$\frac{\partial l}{\partial \lambda} = 0 \Leftrightarrow 1 - \lambda \cdot \bar{x} = 0 \Leftrightarrow \hat{\lambda}_{MLE} = \frac{1}{\bar{x}}$$

$$E[X] = \frac{1}{\lambda} = \bar{x} \Rightarrow \hat{\lambda}_{MM} = \frac{1}{\bar{x}}$$

$$\text{Deci, } \hat{\lambda}_{MM} = \hat{\lambda}_{MLE} = \frac{1}{\bar{x}}$$

v) $X \sim \Pi(\alpha, p)$

Metoda momentelor:

$$E[X] = \alpha p = \bar{x}$$

$$\text{Var}(X) = \alpha p^2 = s^2$$

$$\left. \begin{aligned} E[X] &= \alpha p = \bar{x} \\ \text{Var}(X) &= \alpha p^2 = s^2 \end{aligned} \right\} \Rightarrow \begin{aligned} \hat{p}_{MM} &= \frac{s^2}{\bar{x}} \\ \hat{\alpha}_{MM} &= \frac{\bar{x}^2}{s^2} \end{aligned}$$

Metoda verosimilității maxime:

$$l = (\alpha - 1) \sum_{j=1}^m \ln x_j - \frac{1}{p} \sum_{j=1}^m x_j - m \alpha \ln p - m \ln \Pi(x)$$

$$\frac{\partial l}{\partial p} = \frac{1}{p^2} \sum_{j=1}^m x_j - \frac{m \alpha}{p} = \frac{1}{p^2} \cdot m (\bar{x} - \alpha \cdot p)$$

$$\frac{\partial l}{\partial p} = 0 \Rightarrow \bar{x} = \alpha \cdot p \Rightarrow \hat{p}_{MLE} = \frac{\bar{x}}{\alpha}$$

$$\frac{\partial l}{\partial \alpha} = \sum_{j=1}^m \ln x_j - m \ln p - m \cdot \frac{1}{p(x)} \cdot p'(x) \Rightarrow \hat{\alpha}_{MLE} \text{ nu poate fi calculat numeric.}$$

vi) $X \sim \text{Beta}(\alpha, \beta)$

Metoda momentelor:

$$\left. \begin{aligned} E[X] &= \frac{\alpha}{\alpha + \beta} = \bar{x} \Rightarrow \alpha = \bar{x} \alpha + \bar{x} \beta \Rightarrow \beta = \frac{\alpha(1 - \bar{x})}{\bar{x}} \\ \text{Var}(X) &= \frac{\alpha \beta}{(\alpha + \beta)^2 (\alpha + \beta + 1)} = s^2 \end{aligned} \right\} \Rightarrow$$

$$\Rightarrow s^2 = \frac{\alpha \cdot \alpha(1 - \bar{x})}{\bar{x} \cdot \left(\alpha + \frac{\alpha(1 - \bar{x})}{\bar{x}} \right)^2 \cdot \left(\alpha + \frac{\alpha(1 - \bar{x})}{\bar{x}} + 1 \right)}$$

$$s^2 = \frac{\alpha^2(1 - \bar{x})}{\bar{x} \cdot \frac{\alpha^2}{\bar{x}^2} \cdot \frac{\alpha + \bar{x}}{\bar{x}}} = \frac{\bar{x}(1 - \bar{x})}{\alpha + \bar{x}} \Rightarrow$$

$$\Rightarrow \alpha = \frac{\bar{x}(1 - \bar{x})}{s^2} - \bar{x} \Rightarrow \hat{\alpha}_{MM} = \bar{x} \left(\frac{1 - \bar{x}}{s^2} - 1 \right)$$

$$\hat{\beta}_{MM} = \left(\frac{1 - \bar{x}}{s^2} - 1 \right) (1 - \bar{x})$$

Metoda verosimilității maxime:

$$l = (\alpha - 1) \sum_{j=1}^m \ln x_j + (\beta - 1) \sum_{j=1}^m \ln(1 - x_j) - m \ln B(\alpha, \beta)$$

Pentru β fixat:

$$\frac{\partial l}{\partial \alpha} = \sum_{j=1}^m \ln x_j - m \cdot \frac{1}{B(\alpha, \beta)} \cdot \frac{\partial B(\alpha, \beta)}{\partial \alpha}$$

Pentru α fixat:

$$\frac{\partial l}{\partial \beta} = \sum_{j=1}^m \ln(1 - x_j) - m \cdot \frac{1}{B(\alpha, \beta)} \cdot \frac{\partial B(\alpha, \beta)}{\partial \beta}$$

Nu putem calcula efectiv, aproximăm prin metode numerice.

b)

```

703 # Volumul esantionului cerut
704 n_volum <- 1000
705
706 # --- 1. REPARTITIA GEOMETRICA ---
707 p_real_geom <- 0.4
708 date_geom <- rgeom(n_volum, prob = p_real_geom)
709
710 # MLE și MM coincid teoretic: 1 / (mean(x) + 1)
711 est_geom <- 1 / (mean(date_geom) + 1)
712
713 cat("GEOMETRICA (p real =", p_real_geom, ")\n")
714 cat("Estimare (MLE/MM):", round(est_geom, 4), "\n\n")
715
716 # --- 2. REPARTITIA BINOMIALA (m=10 încercări) ---
717 p_real_bin <- 0.7
718 m_incercari <- 10
719 date_bin <- rbinom(n_volum, size = m_incercari, prob = p_real_bin)
720
721 # MLE și MM: mean(x) / m
722 est_bin <- mean(date_bin) / m_incercari
723
724 cat("BINOMIALA (p real =", p_real_bin, ")\n")
725 cat("Estimare (MLE/MM):", round(est_bin, 4), "\n\n")
726
727 # --- 3. REPARTITIA GAMMA ---
728 alpha_real <- 2
729 beta_real <- 3 # scale
730 date_gamma <- rgamma(n_volum, shape = alpha_real, scale = beta_real)
731
732 # Metoda Momentelor (MM)
733 mean_g <- mean(date_gamma)
734 var_g <- var(date_gamma)
735 alpha_mm <- mean_g^2 / var_g
736 beta_mm <- var_g / mean_g
737
738 # Verosimilitate Maxima (MLE) - optimizare numerica
739 log_lik_gamma <- function(par) {
740   -sum(dgamma(date_gamma, shape = par[1], scale = par[2], log = TRUE))
741 }
742 res_gamma <- optim(c(1, 1), log_lik_gamma)$par
743
744 cat("GAMMA (alpha =", alpha_real, ", beta =", beta_real, ")\n")
745 cat("MM: alpha =", round(alpha_mm, 4), ", beta =", round(beta_mm, 4), "\n")
746 cat("MLE: alpha =", round(res_gamma[1], 4), ", beta =", round(res_gamma[2], 4), "\n\n")
747
748 # --- 4. REPARTITIA BETA ---
749 a_real <- 2
750 b_real <- 5
751 date_beta <- rbeta(n_volum, shape1 = a_real, shape2 = b_real)
752
753 # Metoda Momentelor (MM)
754 m1 <- mean(date_beta)
755 v1 <- var(date_beta)
756 a_mm <- m1 * (m1 * (1 - m1) / v1 - 1)
757 b_mm <- (1 - m1) * (m1 * (1 - m1) / v1 - 1)
758
759 # Verosimilitate Maxima (MLE)
760 log_lik_beta <- function(par) {
761   -sum/dbeta(date_beta, shape1 = par[1], shape2 = par[2], log = TRUE))
762 }
763 res_beta <- optim(c(1, 1), log_lik_beta)$par

```

```

764
765 cat("BETA (alpha =", a_real, ", beta =", b_real, ")\n")
766 cat("MM: alpha =", round(a_mm, 4), ", beta =", round(b_mm, 4), "\n")
767 cat("MLE: alpha =", round(res_beta[1], 4), ", beta =", round(res_beta[2], 4), "\n")
768

```

```

770 # --- 5. REPARTITIA POISSON ---
771 lambda_real_pois <- 5
772 # Generam date conform distributiei
773 date_pois <- rpois(n_volum, lambda = lambda_real_pois)
774
775 # MLE si MM coincid: media aritmetica (x_bar)
776 est_pois <- mean(date_pois)
777
778 cat("POISSON (lambda real =", lambda_real_pois, ")\n")
779 cat("Estimare (MLE/MM):", round(est_pois, 4), "\n\n")
780
781
782 # --- 6. REPARTITIA EXPONENTIALA ---
783 lambda_real_exp <- 0.5
784 date_exp <- rexp(n_volum, rate = lambda_real_exp)
785
786 # MLE si MM coincid: 1 / media aritmetica (1/x_bar)
787 est_exp <- 1 / mean(date_exp)
788
789 cat("EXPONENTIALA (lambda real =", lambda_real_exp, ")\n")
790 cat("Estimare (MLE/MM):", round(est_exp, 4), "\n\n")
791
792
793 # --- 7. REPARTITIA GAMMA (2 parametri) ---
794 # Parametrii: alpha (shape) si beta (scale)
795 alpha_real <- 2
796 beta_real <- 4
797 date_gamma <- rgamma(n_volum, shape = alpha_real, scale = beta_real)
798
799 # Metoda Momentelor (MM) - Formule directe din momentele teoretice
800 mean_g <- mean(date_gamma)
801 var_g <- var(date_gamma)
802 alpha_mm <- mean_g^2 / var_g
803 beta_mm <- var_g / mean_g
804
805 # Verosimilitate Maxima (MLE) - Necesita optimizare numerica
806 # 'par' este vectorul [alpha, beta]
807 log_lik_gamma <- function(par, date) {
808   # Constrangem parametrii sa fie pozitivi
809   if(any(par <= 0)) return(Inf)
810   # Returnam minus log-verosimilitatea pentru minimizare
811   return(-sum(dgamma(date, shape = par[1], scale = par[2], log = TRUE)))
812 }
813
814 res_gamma_mle <- optim(par = c(1, 1), fn = log_lik_gamma, date = date_gamma)$par
815
816 cat("GAMMA (alpha real =", alpha_real, ", beta real =", beta_real, ")\n")
817 cat("MM: alpha =", round(alpha_mm, 4), ", beta =", round(beta_mm, 4), "\n")
818 cat("MLE: alpha =", round(res_gamma_mle[1], 4), ", beta =", round(res_gamma_mle[2], 4), "\n")
819

```

Rezultate. →

```

GEOMETRICA (p real = 0.4 )
> cat("Estimare (MLE/MM):", round(est_geom, 4), "\n\n")
Estimare (MLE/MM): 0.3981

```

→ BINOMIALA (p real = 0.7)

```

> cat("Estimare (MLE/MM):", round(est_bin, 4), "\n\n")
Estimare (MLE/MM): 0.6953

```

→

```

GAMMA (alpha = 2 , beta = 3 )
> cat("MM: alpha =", round(alpha_mm, 4), ", beta =", round(beta_mm, 4), "\n")
MM: alpha = 1.8782 , beta = 3.1887
> cat("MLE: alpha =", round(res_gamma[1], 4), ", beta =", round(res_gamma[2], 4), "\n\n")
MLE: alpha = 1.9283 , beta = 3.1062

```

→ BETA (alpha = 2 , beta = 5)

```

> cat("MM: alpha =", round(a_mm, 4), ", beta =", round(b_mm, 4), "\n")
MM: alpha = 1.8459 , beta = 4.8271
> cat("MLE: alpha =", round(res_beta[1], 4), ", beta =", round(res_beta[2], 4), "\n")
MLE: alpha = 1.8927 , beta = 4.9439
>

```



```

-> cat("POISSON (lambda real = 5 )", lambda_real_pois, "\n")
> cat("Estimare (MLE/MM):", round(est_pois, 4), "\n\n")
Estimare (MLE/MM): 5.123

-> cat("EXPONENTIALA (lambda real = 0.5 )", lambda_real_exp, "\n")
EXPONENTIALA (lambda real = 0.5 )
> cat("Estimare (MLE/MM):", round(est_exp, 4), "\n\n")
Estimare (MLE/MM): 0.4974

-> cat("GAMMA (alpha real = 2 , beta real = 4 )", alpha_real_gamma, "\n")
GAMMA (alpha real = 2 , beta real = 4 )
> cat("MM: alpha =", round(alpha_mm, 4), ", beta =", round(beta_mm, 4), "\n")
MM: alpha = 2.1407 , beta = 3.876
> cat("MLE: alpha =", round(res_gamma_mle[1], 4), ", beta =", round(res_gamma_mle[2], 4), "\n")
MLE: alpha = 2.1438 , beta = 3.8719
>

```

4. Inegalitatea Berry-Esseen și acuratețea aproximării normale

Se consideră un eșantion i.i.d. $X_1, \dots, X_n$ cu:

$$E(X_i) = \mu, \quad \text{Var}(X_i) = \sigma^2, \quad \rho = E|X_i - \mu|^3 < \infty.$$

Inegalitatea Berry-Esseen afirmă că:

$$\sup_x |F_n(x) - \Phi(x)| \leq C \frac{\rho}{\sigma^3 \sqrt{n}}$$

unde F_n este funcția de repartiție a variabilei $Z_n = \frac{\sqrt{n}(X_n - \mu)}{\sigma}$, iar Φ este funcția de repartiție a normalei standard.

a) Calculați ρ pentru repartițiile: Binomială, Geometrică, Poisson, Uniformă (discret și continuu), Exponențială, Gamma și Beta.

b) Reprezentați grafic funcțiile obținute la punctul a).

c) Construiți în R:

- funcție care calculează ρ ;
- funcție care calculează marginea Berry-Esseen;
- funcție care aproximează numeric $\sup_x |F_n(x) - \Phi(x)|$.

În practică, supremul poate fi aproximat numeric ca maxim al diferenței pe o grilă fină de valori, aleasă în funcție de cuantilele distribuției normale.

d) Construiți un dataframe care să conțină marginile date de inegalitatea Berry-Esseen pentru repartițiile de mai sus, pentru diferite dimensiuni al eșantionului n alese de voi.

e) Ilustrați grafic evoluția diferenței $|F_n(x) - \Phi(x)|$ pentru un interval relevant de valori și discutați rezultatele obținute.

a) 1) Binomială

$$\begin{aligned}
 X &\sim \text{Bin}(m, p) \\
 \text{support } k &= 0, 1, \dots, m \\
 \mu &= mp, \sigma^2 = mp(1-p) \\
 f &= \sum_{k=0}^m |k - mp|^3 \binom{m}{k} p^k (1-p)^{m-k}
 \end{aligned}$$

2) Geometrică

$$\begin{aligned}
 X &\sim \text{Geom}(p) \\
 \text{A) support } &\{1, 2, 3, \dots\} \\
 P(X=k) &= p(1-p)^{k-1} \\
 \mu &= \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2} \\
 \text{B) support } &\{0, 1, 2, \dots\} \\
 P(X=k) &= p(1-p)^k \\
 \mu &= \frac{1-p}{p}, \sigma^2 = \frac{1-p}{p^2}
 \end{aligned}$$

în ambele cazuri:

$$f = \sum_k |k - \mu|^3 P(X=k)$$

3) Poisson

$$X \sim \text{Poisson}(\lambda)$$

support $k = 0, 1, 2, \dots$

$$\mu = \lambda, \sigma^2 = \lambda$$

$$P = \sum_{k=0}^{\infty} |k - \lambda|^3 e^{-\lambda} \frac{\lambda^k}{k!}$$

4) Uniformă discretă

dacă X e uniformă pe mulțimea $\{a, a+1, \dots, b\}$ cu $N = b - a + 1$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{N^2 - 1}{12}$$

$$P = \frac{1}{N} \sum_{k=a}^b |k - \mu|^3$$

5) Uniformă continuă

$$X \sim U(a, b)$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

$$P = E|X - \mu|^3 = 2 \int_a^{\mu} (\mu - x)^3 \frac{1}{b-a} dx \quad \text{s.v. } \mu = \mu - x$$

$$x=a \Rightarrow \mu = \mu - a = \frac{a+b}{2} - a = \frac{b-a}{2}$$

$$x=\mu \Rightarrow \mu = \mu - \mu = 0$$

$$P = \frac{2}{b-a} \int_a^{\mu} u^3 du$$

$$= \frac{2}{b-a} \cdot \frac{(\frac{b-a}{2})^4}{4} = \frac{(b-a)^3}{32}$$

6) Exponențială

$$X \sim \text{Exp}(\lambda)$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

$$P = \int_0^{\infty} |x - \mu|^3 \lambda e^{-\lambda x} dx = \underbrace{\int_0^{\mu} (\mu - x)^3 \lambda e^{-\lambda x} dx}_{J_1} + \underbrace{\int_{\mu}^{\infty} (x - \mu)^3 \lambda e^{-\lambda x} dx}_{J_2}$$

$$\int_0^{\mu} (\mu - x)^3 \lambda e^{-\lambda x} dx \quad \text{s.v. } \mu = \mu - x \Rightarrow x = \mu - u$$

$$dx = -du$$

$$x=0 \Rightarrow u = \mu$$

$$x=\mu \Rightarrow u = 0$$

$$\int_{\mu}^0 u^3 \lambda e^{-\lambda(\mu - u)} (-du) = e^{-\lambda \mu} \int_0^{\mu} u^3 \lambda e^{\lambda u} du = e^{-\lambda \mu} \int_0^{\mu} u^3 \lambda e^{\lambda u} du$$

$$\text{s.v. } w = \lambda u \Rightarrow u = \frac{w}{\lambda} \quad du = \frac{1}{\lambda} dw$$

$$u=0 \Rightarrow w=0$$

$$u=\mu \Rightarrow w = \lambda \mu = \lambda \cdot \frac{1}{\lambda} = 1$$

$$e^{-1} \int_0^1 \left(\frac{w}{\lambda}\right)^3 \lambda e^w \frac{dw}{\lambda} = \frac{e^{-1}}{\lambda^3} \int_0^1 w^3 e^w dw$$

$$\int_0^1 w^3 e^w dw = \int_0^1 w^3 (e^w)' dw = w^3 e^w \Big|_0^1 - \int_0^1 3w^2 e^w dw$$

$$= 1 - 3 \int_0^1 w^2 (e^w)' dw = 1 - 3 \left(w^2 e^w \Big|_0^1 - \int_0^1 2w e^w dw \right) =$$

$$= 1 - 3 \left(1 - 2 \int_0^1 w (e^w)' dw \right) = 1 - 3 + 6 \left(w e^w \Big|_0^1 - \int_0^1 e^w dw \right)$$

$$= 1 - 3 + 6 - 6 + 6 = 4$$

$$J_1 = \frac{e^{-1}}{\lambda^3} (4 - 2) = \frac{2}{e \lambda^3} - \frac{2}{\lambda^3}$$

J_2 :

$$\int_{\mu}^{\infty} (x - \mu)^3 \lambda e^{-\lambda x} dx \quad \text{s.v. } t = x - \mu \Rightarrow x = t + \mu \quad dx = dt$$

$$x=\mu \Rightarrow t=0$$

$$x=\infty \Rightarrow t=\infty$$

$$\int_0^{\infty} t^3 \lambda e^{-\lambda(t + \mu)} dt = e^{-\lambda \mu} \int_0^{\infty} t^3 \lambda e^{-\lambda t} dt \quad \text{s.v. } s = \lambda t$$

$$t = \frac{s}{\lambda}$$

$$e^{-1} \int_0^{\infty} \left(\frac{s}{\lambda}\right)^3 \lambda e^{-s} \frac{ds}{\lambda} = \frac{1}{\lambda^3} \int_0^{\infty} s^3 e^{-s} ds = \frac{1}{\lambda^3} \cdot 3! \quad dt = \frac{ds}{\lambda}$$

$$J_2 = \frac{6}{\lambda^3 e}$$

$$P = J_1 + J_2 = \frac{2}{e \lambda^3} - \frac{2}{\lambda^3} + \frac{6}{e \lambda^3} = \frac{8}{e \lambda^3} - \frac{2}{\lambda^3} = \frac{1}{\lambda^3} \left(\frac{8}{e} - 2 \right)$$

7) Gamma

$$X \sim \Gamma(\alpha, \beta)$$

$$\mu = \frac{\alpha}{\beta}, \quad \sigma^2 = \frac{\alpha}{\beta^2}$$

$$\rho = \int_0^\infty |x - \mu|^3 f_\Gamma(x) dx$$

Se calculează numeric cu integrate()

8) Beta

$$X \sim \text{Beta}(\alpha, \beta)$$

$$\mu = \frac{\alpha}{\alpha + \beta}$$

$$\sigma^2 = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

$$\rho = \int_0^1 |x - \mu|^3 f_{\text{Beta}}(x) dx$$

Se calculează numeric cu integrate()

b)

```
# calculul lui rho = E|X - mu|^3
# pentru mai multe repartitii (discrete si continue)
# Ideea:
# - Discret: rho = sum_x |x - mu|^3 * P(X=x)
# - Continuu: rho = integral |x - mu|^3 * f(x) dx
#####

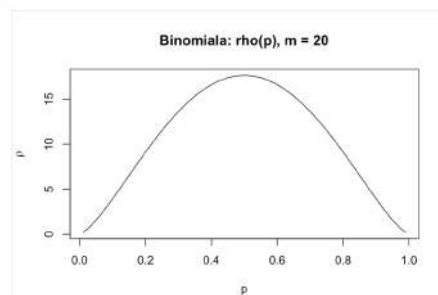
# 1) Functii ajutatoare
#####

# rho pentru o variabila discreta data prin:
# xs = vector de valori posibile (suport)
# ps = vector de probabilitati P(X=xs[i])
# mu = media (E[X])
rho_discrete <- function(xs, ps, mu) {
  sum(abs(xs - mu)^3 * ps)
}

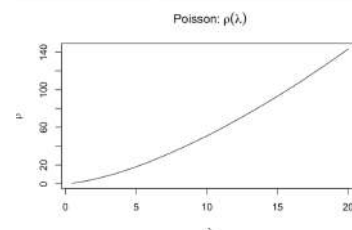
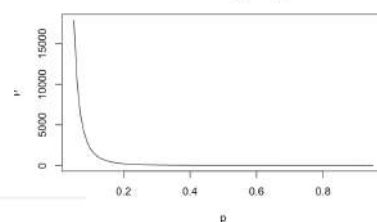
# rho pentru o variabila continua data prin:
# f = densitatea f(x)
# mu = media (E[X])
# lower, upper = capete de integrare (pot fi -Inf, Inf)
rho_continuu <- function(f, mu, lower, upper) {
  # integrate() vrea o functie care primeste vector x
  g <- function(x) abs(x - mu)^3 * f(x)

  # integrate returneaza o lista; valoarea e in $value
  integrate(g, lower = lower, upper = upper, rel.tol = 1e-10)$value
}

# Pentru distributii discrete cu suport infinit (Poisson, Geom),
# facem trunciere numerica: luam xs = 0..K, unde K e un cuantila mare
# (aproape de 1), de ex. q*(1 - eps).
#
# eps mic => K mai mare => aproximare mai buna, dar mai lenta.
default_eps <- 1e-12
```



Geometrica (R): rho(p)



define

```
855 # 2) Distributii discrete
856 #####
857
858 # 2.1) Binomiala X ~ Bin(m, p), suport {0,1,...,m}
859 rho_binom <- function(m, p) {
860   xs <- 0:m
861   ps <- dbinom(xs, size = m, prob = p)
862   mu <- m * p
863   rho_discrete(xs, ps, mu)
864 }
865
866 # 2.2) Geometrica (R): X ~ Geom(p), suport {0,1,2,...}
867 # Interpretare R: X = nr. de esecuri inainte de primul succes
868 # P(X=k) = (1-p)^k * p
869 # Media: mu = (1-p)/p
870 rho_geom <- function(p, eps = default_eps) {
871   # alegem un K suficient de mare incat P(X <= K) ~ 1 - eps
872   K <- qgeom(1 - eps, prob = p) # cuantila pentru suport 0..Inf
873
874   xs <- 0:K
875   ps <- dgeom(xs, prob = p)
876   mu <- (1 - p) / p
877
878   rho_discrete(xs, ps, mu)
879 }
880
881 # 2.3) Poisson X ~ Pois(lambda), suport {0,1,2,...}
882 # Media: mu = lambda
883 rho_pois <- function(lambda, eps = default_eps) {
884   K <- qpois(1 - eps, lambda = lambda)
885
886   xs <- 0:K
887   ps <- dpois(xs, lambda = lambda)
888   mu <- lambda
889
890   rho_discrete(xs, ps, mu)
891 }
```

```
950 # 1) Binomiala: rho(p) pentru m fix
951 #####
952 m_fixed <- 20
953 p_grid <- seq(0.01, 0.99, by = 0.01) # evitam 0 si 1 (degenerate)
954
955 rho_binom_vals <- sapply(p_grid, function(p) rho_binom(m = m_fixed, p = p))
956
957 plot(p_grid, rho_binom_vals, type = "l",
958      xlab = "p", ylab = expression(rho),
959      main = paste0("Binomiala: rho(p), m = ", m_fixed))
960
961 #####
962 # 2) Geometrica (conventia R): rho(p)
963 #####
964 p_grid_geom <- seq(0.05, 0.95, by = 0.01) # evitam p prea mic (coada foarte lunga)
965
966 rho_geom_vals <- sapply(p_grid_geom, function(p) rho_geom(p))
967
968 plot(p_grid_geom, rho_geom_vals, type = "l",
969      xlab = "p", ylab = expression(rho),
970      main = "Geometrica (R): rho(p)")
971
972 #####
973 # 3) Poisson: rho(lambda)
974 #####
975 lambda_grid <- seq(0.5, 20, by = 0.5)
976
977 rho_pois_vals <- sapply(lambda_grid, function(lam) rho_pois(lam))
978
979 plot(lambda_grid, rho_pois_vals, type = "l",
980      xlab = expression(lambda), ylab = expression(rho),
981      main = expression(paste("Poisson: ", rho(lambda))))
982
```

grafice

```

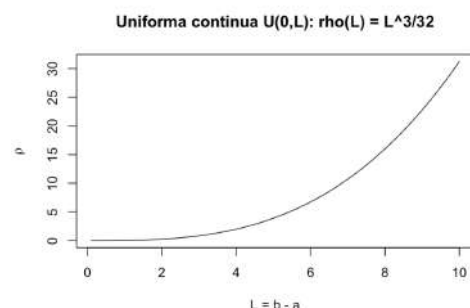
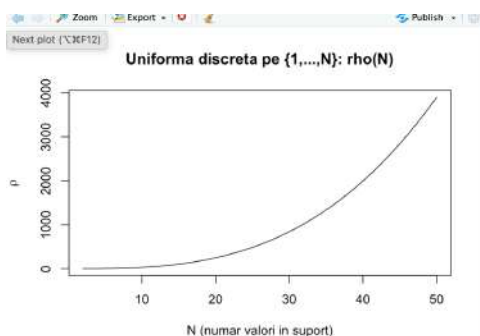
894 # 2.4) Uniforma discreta pe {a, a+1, ..., b}
895 # Media: mu = (a+b)/2
896 rho_unif_disc <- function(a, b) {
897   xs <- a:b
898   N <- length(xs)
899   ps <- rep(1 / N, N)
900   mu <- (a + b) / 2
901
902   rho_discrete(xs, ps, mu)
903 }
904
905 #####
906 # 3) Distributii continue
907 #####
908
909 # 3.1) Uniforma continua X ~ U(a, b)
910 # Exista formula inchisa: rho = (b - a)^3 / 32
911 # 0 dau direct (e exact), dar poti verifica si numeric cu integrate().
912 rho_unif_cont <- function(a, b) {
913   (b - a)^3 / 32
914 }
915
916 # 3.2) Exponentiala X ~ Exp(rate = lambda)
917 # Media: mu = 1/lambda, suport [0, Inf)
918 # Facem numeric cu integrate().
919 rho_exp <- function(lambda) {
920   mu <- 1 / lambda
921   f <- function(x) dexp(x, rate = lambda)
922   rho_continuous(f, mu, lower = 0, upper = Inf)
923 }
924
925 # 3.3) Gamma X ~ Gamma(shape = alpha, rate = beta)
926 # (Atentie: in R, dgamma foloseste shape si rate; scale = 1/rate)
927 # Media: mu = alpha / beta, suport [0, Inf)
928 rho_gamma <- function(alpha, beta) {
929   mu <- alpha / beta
930   f <- function(x) dgamma(x, shape = alpha, rate = beta)
931   rho_continuous(f, mu, lower = 0, upper = Inf)
932 }
933
934 # 3.4) Beta X ~ Beta(alpha, beta), suport [0, 1]
935 # Media: mu = alpha / (alpha + beta)
936 rho_beta <- function(alpha, beta) {
937   mu <- alpha / (alpha + beta)
938   f <- function(x) dbeta(x, shape1 = alpha, shape2 = beta)
939   rho_continuous(f, mu, lower = 0, upper = 1)
940 }
941
942

```

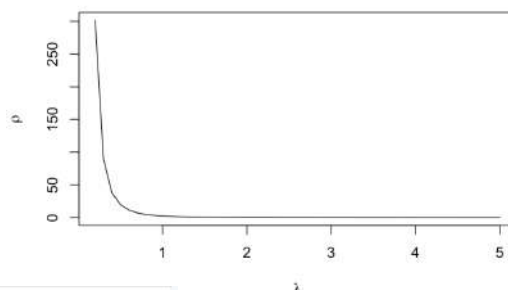
```

894 # 4) Uniforma discreta: rho in functie de "ltime" N
895 # Exemplu: suport {1,2,...,N} (a=1, b=N)
896 #####
897 N_grid <- 2:50 # N=1 ar fi degenerata (rho=0)
898 rho_unif_disc_vals <- sapply(N_grid, function(N) rho_unif_disc(a = 1, b = N))
899
900 plot(N_grid, rho_unif_disc_vals, type = "l",
901      xlab = "N (numar valori in suport)", ylab = expression(rho),
902      main = "Uniforma discreta pe {1,...,N}: rho(N)")
903
904 #####
905 # 5) Uniforma continua: rho in functie de lungimea intervalului L = b-a
906 # Exemplu: U(0, L)
907 #####
908 L_grid <- seq(0.1, 10, by = 0.1)
909 rho_unif_cont_vals <- (L_grid^3) / 32 # formula exacta, deoarece a=0, b=L => (b-a)^3/32
1000
1001 plot(L_grid, rho_unif_cont_vals, type = "l",
1002      xlab = "L = b - a", ylab = expression(rho),
1003      main = "Uniforma continua U(0,L): rho(L) = L^3/32")
1004
1005 #####
1006 # 6) Exponentiala: rho(lambda) (rate lambda)
1007 #####
1008 lambda_grid_exp <- seq(0.2, 5, by = 0.1) # lambda>0
1009
1010 rho_exp_vals <- sapply(lambda_grid_exp, function(lam) rho_exp(lam))
1011
1012 plot(lambda_grid_exp, rho_exp_vals, type = "l",
1013      xlab = expression(lambda), ylab = expression(rho),
1014      main = expression(paste("Exponentiala: ", rho(lambda), " (rate)")))
1015
1016 #####
1017 # 7) Gamma: rho(alpha) cu rate beta fix
1018 #####
1019 beta_fixed <- 1
1020 alpha_grid <- seq(0.5, 10, by = 0.25) # alpha>0
1021
1022 rho_gamma_vals <- sapply(alpha_grid, function(a) rho_gamma(alpha = a, beta = beta_fixed))
1023
1024 plot(alpha_grid, rho_gamma_vals, type = "l",
1025      xlab = expression(alpha), ylab = expression(rho),
1026      main = bquote("Gamma: " ~ rho(alpha) ~ ", rate" ~ beta == .(beta_fixed)))
1027
1028
1029 # 8) Beta: rho(alpha) cu beta fix (sau invers)
1030 #####
1031 beta_fixed_beta <- 2
1032 alpha_grid_beta <- seq(0.5, 10, by = 0.25)
1033
1034 rho_beta_vals <- sapply(alpha_grid_beta, function(a) rho_beta(alpha = a, beta = beta_fixed_beta))
1035
1036 plot(alpha_grid_beta, rho_beta_vals, type = "l",
1037      xlab = expression(alpha), ylab = expression(rho),
1038      main = bquote("Beta: " ~ rho(alpha) ~ ", " ~ beta == .(beta_fixed_beta)))
1039

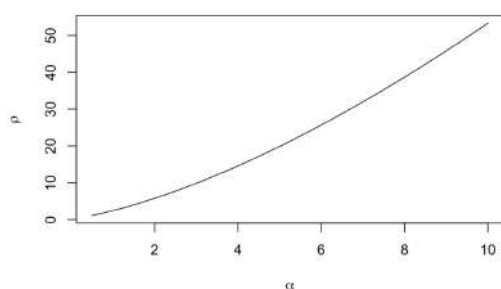
```



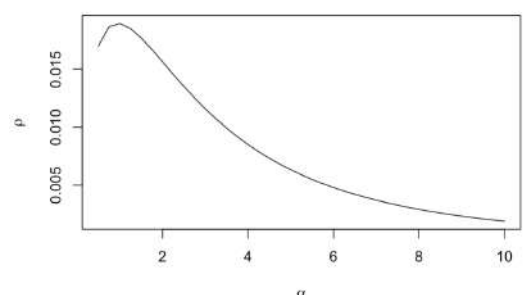
Exponentiala: $\rho(\lambda)$ (rate)



Gamma: $\rho(\alpha)$, rate $\beta = 1$



Beta: $\rho(\alpha)$, $\beta = 2$



c)

```

1051 # 2) Marginea Berry-Esseen
1052 #####
1053 # Inegalitatea Berry-Esseen (forma din proiect):
1054 # sup_x |F_n(x) - Phi(x)| <= C * rho / (sigma^3 * sqrt(n))
1055 #
1056 # - n = volumul esantionului
1057 # - rho = E|X - mu|^3
1058 # - sigma = sd(X)
1059 # - C = constanta (in teorie exista o constanta universală; in exercitiul 5 o estimati)
1060 berry_esseen_bound <- function(n, rho, sigma, C = 0.56) {
1061   if (any(n <= 0)) stop("n trebuie sa fie pozitiv.")
1062   if (any(sigma <= 0)) stop("sigma trebuie sa fie > 0.")
1063   C * rho / (sigma^3 * sqrt(n))
1064 }
1065 }
1066

```

```

1068 #####
1069 # 3) Aproximare numerică a sup_x |F_n(x) - Phi(x)|
1070 #####
1071 # In practica, supremumul se aproximeaza pe o grila fina de x.
1072 # Grila o alegem folosind cuantilele normale (cum zice cerinta):
1073 # x_grid = qnorm(seq(eps, 1-eps, ...))
1074 make_x_grid <- function(eps = 1e-4, n_points = 3000) {
1075   qnorm(seq(eps, 1 - eps, length.out = n_points))
1076 }
1077
1078 # Functie generica: primeste Fn(x) si o grila x_grid,
1079 # returneaza max_x |Fn(x) - Phi(x)| pe grila.
1080 sup_diff_numeric <- function(Fn, x_grid) {
1081   diffs <- abs(Fn(x_grid) - pnorm(x_grid))
1082   max(diffs)
1083 }
1084

```

Fold code

```

1087 # 4) Functii Fn(x) pentru Z_n
1088 #####
1089 # Zn = sqrt(n) * (Xbar - mu) / sigma
1090 # Fn(x) = P(Zn <= x) = P(Xbar <= mu + x*sigma/sqrt(n))
1091
1092 # 4.1) Fn exact pentru Binomiala:
1093 # Daca Xi ~ Bin(m,p), atunci Sn = sum Xi ~ Bin(n*m, p).
1094 Fn_Zn_binom <- function(x, n, m, p) {
1095   mu <- m * p
1096   sigma <- sqrt(m * p * (1 - p))
1097
1098   # prag pentru media:
1099   t_bar <- mu + x * sigma / sqrt(n)
1100   # prag pentru suma:
1101   t_sum <- n * t_bar
1102
1103   # Sn discret => folosim floor
1104   pbinom(floor(t_sum), size = n * m, prob = p)
1105 }
1106
1107 # 4.2) Fn exact pentru Poisson:
1108 # Daca Xi ~ Pois(lambda), atunci Sn ~ Pois(n*lambda).
1109 Fn_Zn_pois <- function(x, n, lambda) {
1110   mu <- lambda
1111   sigma <- sqrt(lambda)
1112
1113   t_bar <- mu + x * sigma / sqrt(n)
1114   t_sum <- n * t_bar
1115
1116   ppois(floor(t_sum), lambda = n * lambda)
1117 }
1118

```

```

1121 # 5) Varianta universală: Fn(x) prin SIMULARE (merge pentru orice distributie)
1122 #####
1123 # Ideea:
1124 # - simulezi M replicari ale lui Zn
1125 # - Fn(x) se aproximeaza cu CDF empirica: mean(Zn_sim <= x)
1126 #
1127 # rX = o functie care genereaza n valori din distributia lui X (ex: rpois, rbinom etc.)
1128 # mu, sigma = parametrii teoretici ai lui X (ca sa standardizam corect)
1129 Fn_Zn_sim <- function(x, n, rX, mu, sigma, M = 20000) {
1130   # generam M valori ale lui Zn
1131   Z <- numeric(M)
1132   for (i in 1:M) {
1133     X <- rX(n) # esantion de marime n
1134     Xbar <- mean(X)
1135     Z[i] <- sqrt(n) * (Xbar - mu) / sigma
1136   }
1137
1138   # CDF empirica in punctele x (vectorizat cu sapply)
1139   sapply(x, function(xx) mean(Z <= xx))
1140 }

```

d)

```

# set.seed(123)
#
# n_values <- c(10, 20, 30, 50, 100, 200)
#
# C_const <- 0.56
# Scenarii: distributie + parametri + mu, sigma, rho
scenarios <- list(
  list(name = "Binom(m=20,p=0.3)",
    mu = 20 * 0.3,
    sigma = sqrt(20 * 0.3 * 0.7),
    rho = rho_binom(20, 0.3)),
  # Geometrica in R: suport {0,1,2,...} (failures)
  list(name = "Geom(p=0.3) [R failures]",
    mu = (1 - 0.3) / 0.3,
    sigma = sqrt((1 - 0.3) / (0.3^2)),
    rho = rho_geom(0.3)),

```



```

1165 list(name = "Pois(lambda=5)",
1166       mu = 5,
1167       sigma = sqrt(5),
1168       rho = rho_pois(5)),
1169
1170 list(name = "UnifDisc(0..9)",
1171       mu = (0 + 9) / 2,
1172       sigma = sqrt((10^2 - 1) / 12), # N = 10 valori (0..9)
1173       rho = rho_unif_disc(0, 9)),
1174
1175 list(name = "UnifCont(0,1)",
1176       mu = 0.5,
1177       sigma = 1 / sqrt(12),
1178       rho = rho_unif_cont(0, 1)),
1179
1180 list(name = "Exp(lambda=2)",
1181       mu = 1 / 2,
1182       sigma = 1 / 2,
1183       rho = rho_exp(2)),
1184
1185 list(name = "Gamma(alpha=2,beta=1) [shape=2,rate=1]",
1186       mu = 2 / 1,
1187       sigma = sqrt(2) / 1, # sd = sqrt(shape)/rate
1188       rho = rho_gamma(2, 1)),
1189
1190 list(name = "Beta(alpha=2,beta=5)",
1191       mu = 2 / (2 + 5),
1192       sigma = sqrt(2 * 5 / ((2 + 5)^2 * (2 + 5 + 1))),
1193       rho = rho_beta(2, 5)),
1194 )
1195
1196 # Construiam dataframe-ul final
1197
1198 df_BE <- do.call(rbind, lapply(scenarios, function(sc) {
1199   data.frame(
1200     dist = sc$name,
1201     n = n_values,
1202     mu = sc$mu,
1203     sigma = sc$sigma,
1204     rho = sc$rho,
1205     C = C_const,
1206     BE_bound = berry_esseen_bound(
1207       n = n_values,
1208       rho = sc$rho,
1209       sigma = sc$sigma,
1210       C = C_const
1211     )
1212   )
1213 })))
1214
1215 # Afisare tabel
1216 print(df_BE)

```

```

> # Afisare tabel
> print(df_BE)

```

	dist	n	mu	sigma	rho	C	BE_bound
1	Binom(n=20,p=0.3)	10	6.0000000	2.0493902	13.60968542	0.56	0.28000644
2	Binom(n=20,p=0.3)	20	6.0000000	2.0493902	13.60968542	0.56	0.19799445
3	Binom(n=20,p=0.3)	30	6.0000000	2.0493902	13.60968542	0.56	0.16166179
4	Binom(n=20,p=0.3)	50	6.0000000	2.0493902	13.60968542	0.56	0.12522269
5	Binom(n=20,p=0.3)	100	6.0000000	2.0493902	13.60968542	0.56	0.08545481
6	Binom(n=20,p=0.3)	200	6.0000000	2.0493902	13.60968542	0.56	0.06151134
7	Geom(p=0.3) [R failures]	10	2.3333333	2.7886668	52.702740347	0.56	0.43026689
8	Geom(p=0.3) [R failures]	20	2.3333333	2.7886668	52.702740347	0.56	0.30424464
9	Geom(p=0.3) [R failures]	30	2.3333333	2.7886668	52.702740347	0.56	0.24841471
10	Geom(p=0.3) [R failures]	50	2.3333333	2.7886668	52.702740347	0.56	0.19242120
11	Geom(p=0.3) [R failures]	100	2.3333333	2.7886668	52.702740347	0.56	0.13606234
12	Geom(p=0.3) [R failures]	200	2.3333333	2.7886668	52.702740347	0.56	0.09621060
13	Pois(lambda=5)	10	5.0000000	2.2360680	18.141804114	0.56	0.28735152
14	Pois(lambda=5)	20	5.0000000	2.2360680	18.141804114	0.56	0.20318821
15	Pois(lambda=5)	30	5.0000000	2.2360680	18.141804114	0.56	0.16590248
16	Pois(lambda=5)	50	5.0000000	2.2360680	18.141804114	0.56	0.12850750
17	Pois(lambda=5)	100	5.0000000	2.2360680	18.141804114	0.56	0.09086853
18	Pois(lambda=5)	200	5.0000000	2.2360680	18.141804114	0.56	0.06425375
19	UnifDisc(0..9)	10	4.5000000	2.8722813	30.625000000	0.56	0.2286701
20	UnifDisc(0..9)	20	4.5000000	2.8722813	30.625000000	0.56	0.16183342
21	UnifDisc(0..9)	30	4.5000000	2.8722813	30.625000000	0.56	0.13213663
22	UnifDisc(0..9)	50	4.5000000	2.8722813	30.625000000	0.56	0.10235244
23	UnifDisc(0..9)	100	4.5000000	2.8722813	30.625000000	0.56	0.07237410
24	UnifDisc(0..9)	200	4.5000000	2.8722813	30.625000000	0.56	0.05117622
25	UnifCont(0,1)	10	0.5000000	0.2886751	0.031250000	0.56	0.2304347
26	UnifCont(0,1)	20	0.5000000	0.2886751	0.031250000	0.56	0.1626530
27	UnifCont(0,1)	30	0.5000000	0.2886751	0.031250000	0.56	0.13281566
28	UnifCont(0,1)	50	0.5000000	0.2886751	0.031250000	0.56	0.10278757
29	UnifCont(0,1)	100	0.5000000	0.2886751	0.031250000	0.56	0.07274613
30	UnifCont(0,1)	200	0.5000000	0.2886751	0.031250000	0.56	0.05143928
31	Exp(lambda=2)	10	0.5000000	0.5000000	0.301819162	0.56	0.42758732
32	Exp(lambda=2)	20	0.5000000	0.5000000	0.301819162	0.56	0.30234990
33	Exp(lambda=2)	30	0.5000000	0.5000000	0.301819162	0.56	0.24686766
34	Exp(lambda=2)	50	0.5000000	0.5000000	0.301819162	0.56	0.19122286
35	Exp(lambda=2)	100	0.5000000	0.5000000	0.301819162	0.56	0.13521498
36	Exp(lambda=2)	200	0.5000000	0.5000000	0.301819162	0.56	0.09516143
37	Gamma(alpha=2,beta=1) [shape=2,rate=1]	10	2.0000000	1.4142136	5.744140393	0.56	0.35964007
38	Gamma(alpha=2,beta=1) [shape=2,rate=1]	20	2.0000000	1.4142136	5.744140393	0.56	0.25430389
39	Gamma(alpha=2,beta=1) [shape=2,rate=1]	30	2.0000000	1.4142136	5.744140393	0.56	0.20763829
40	Gamma(alpha=2,beta=1) [shape=2,rate=1]	50	2.0000000	1.4142136	5.744140393	0.56	0.16083593
41	Gamma(alpha=2,beta=1) [shape=2,rate=1]	100	2.0000000	1.4142136	5.744140393	0.56	0.11372818
42	Gamma(alpha=2,beta=1) [shape=2,rate=1]	200	2.0000000	1.4142136	5.744140393	0.56	0.08041797
43	Beta(alpha=2,beta=5)	10	0.2857143	0.1597191	0.006328599	0.56	0.2750826
44	Beta(alpha=2,beta=5)	20	0.2857143	0.1597191	0.006328599	0.56	0.19449556
45	Beta(alpha=2,beta=5)	30	0.2857143	0.1597191	0.006328599	0.56	0.15800496
46	Beta(alpha=2,beta=5)	50	0.2857143	0.1597191	0.006328599	0.56	0.12300979
47	Beta(alpha=2,beta=5)	100	0.2857143	0.1597191	0.006328599	0.56	0.08698106
48	Beta(alpha=2,beta=5)	200	0.2857143	0.1597191	0.006328599	0.56	0.06150490

rezultat

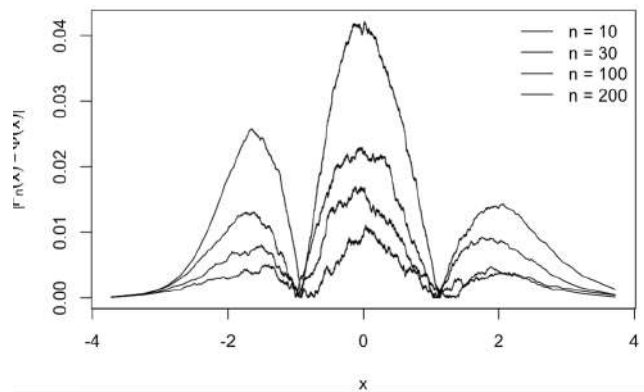
e)

```

1224 # Grila de x (din cuantile normale)
1225 xg <- make_x_grid(eps = 1e-4, n_points = 2000)
1226
1227 # Fix util daca iti apare "figure margins too large"
1228 par(mar = c(4, 4, 3, 1))
1229 # Functie: curba IFn - Phi pentru un Fn dat
1230 diff_curve <- function(Fn, x_grid) {
1231   abs(Fn(x_grid) - pnorm(x_grid))
1232 }
1233
1234 # Simulare RAPIDA pentru Zn + Fn empiric (ecdf)
1235 Zn_sim_fast <- function(n, rX, mu, sigma, M = 30000) {
1236   # genereaza M replicari ale lui Zn rapid, fara for
1237   X <- matrix(rX(n * M), nrow = M, ncol = n)
1238   Z <- sqrt(n) * (rowMeans(X) - mu) / sigma
1239   Z
1240 }
1241
1242 Fn_from_Z <- function(Z) {
1243   ecdf(Z) # returneaza o functie Fn(x)
1244 }
1245
1246 # Functie: plateaza evolutia IFn - Phi pentru mai multe n
1247 # + optional linia Berry-Esseen
1248 plot_diff_evolution <- function(n_values, Fn_builder, title,
1249   add_BE = FALSE, rho = NULL, sigma = NULL, C = 0.56) {
1250   # prima curba (ca sa setam axele)
1251   Fn0 <- Fn_builder(n_values[1])
1252   d0 <- diff_curve(Fn0, xg)
1253
1254   plot(xg, d0, type = "l",
1255        xlab = "x", ylab = expression(abs(F[n](x) - Phi(x))),
1256        main = title)
1257
1258   # restul curbelor
1259   if (length(n_values) > 1) {
1260     for (k in 2:length(n_values)) {
1261       Fn_k <- Fn_builder(n_values[k])
1262       dk <- diff_curve(Fn_k, xg)
1263       lines(xg, dk)
1264     }
1265   }
1266
1267   legend("topright",
1268         legend = paste0("n = ", n_values),
1269         lty = 1, bty = "n")
1270
1271   # optional: linie Berry-Esseen (pentru fiecare n ar fi alta;
1272   # punem cea mai mare (n minim) ca referinta conservatoare)
1273   if (add_BE) {
1274     if (is.null(rho) || is.null(sigma)) stop("Pentru BE trebuie sa dai rho si sigma.")
1275     bound_max <- berry_esseen_bound(n = min(n_values), rho = rho, sigma = sigma, C = C)
1276     abline(h = bound_max, lty = 2)
1277   }
1278
1279   # returnam si un mic rezumat numeric (sup pe grila)
1280   sups <- sapply(n_values, function(nn) {
1281     Fn_n <- Fn_builder(nn)
1282     max(diff_curve(Fn_n, xg))
1283   })
1284   data.frame(n = n_values, sup_on_grid = sups)
1285 }
1286 n_show <- c(10, 30, 100, 200)

```

Exponential (sim): |Fn(x) - Phi(x)| (lambda=2)



```

1304 - #####
1305 # (1) BINOMIALA - Fn exact
1306 - #####
1307 m <- 20; p <- 0.3
1308 rho_b <- rho_binom(m, p)
1309 sigma_b <- sqrt(m * p * (1 - p))
1310
1311 res_binom <- plot_diff_evolution(
1312   n_values = n_show,
1313   Fn_builder = function(n) function(x) Fn_Zn_binom(x, n = n, m = m, p = p),
1314   title = paste0("Binomiala: |Fn(x) - Phi(x)| (m=", m, ", p=", p, ")"),
1315   add_BE = TRUE, rho = rho_b, sigma = sigma_b, C = 0.56
1316 )
1317 print(res_binom)
1318
1319 #####
1320 # (2) POISSON - Fn exact
1321 - #####
1322 lambda <- 5
1323 rho_p <- rho_pois(lambda)
1324 sigma_p <- sqrt(lambda)
1325
1326 res_pois <- plot_diff_evolution(
1327   n_values = n_show,
1328   Fn_builder = function(n) function(x) Fn_Zn_pois(x, n = n, lambda = lambda),
1329   title = paste0("Poisson: |Fn(x) - Phi(x)| (lambda=", lambda, ")"),
1330   add_BE = TRUE, rho = rho_p, sigma = sigma_p, C = 0.56
1331 )
1332 print(res_pois)
1333
1334 #####
1335 # (3) EXEMPLU CONTINUU (EXPONENTIALA) - Fn prin simulare
1336 - #####
1337 lambda_e <- 2
1338 mu_e <- 1 / lambda_e
1339 sigma_e <- 1 / lambda_e
1340 rho_e <- rho_exp(lambda_e)
1341
1342 # aici Fn_builder simuleaza Z o data pentru fiecare n (rapid), apoi foloseste ecdf
1343 res_exp <- plot_diff_evolution(
1344   n_values = n_show,
1345   Fn_builder = function(n) {
1346     Z <- Zn_sim_fast(n, rX = function(k) rexp(k, rate = lambda_e),
1347                       mu = mu_e, sigma = sigma_e, M = 40000)
1348     Fn_from_Z(Z)
1349   },
1350   title = paste0("Exponentiala (sim): |Fn(x) - Phi(x)| (lambda=", lambda_e, ")"),
1351   add_BE = TRUE, rho = rho_e, sigma = sigma_e, C = 0.56
1352 )
1353 print(res_exp)
1354
1355 #####

```

```

> print(res_binom)
      n sup_on_grid
1  10 0.034615844
2  30 0.019875390
3 100 0.010877464
4 200 0.007789626
>

```

```

> print(res_pois)
      n sup_on_grid
1  10 0.037266616
2  30 0.021447105
3 100 0.011808181
4 200 0.008339478
~

```

```

> print(res_exp)
      n sup_on_grid
1  10 0.04282508
2  30 0.02559947
3 100 0.01302207
4 200 0.01406883

```

5. Estimarea constantei din inegalitatea Berry-Esseen

a) Pentru fiecare repartiție studiată, definiți: $C_n = \frac{\sup_x |F_n(x) - \Phi(x)| \sigma^3 \sqrt{n}}{\rho}$.

b) Estimați prin simulare Monte Carlo valorile lui C_n pentru mai multe volume ale eșantionului n (de exemplu $n \in \{30, 50, 100, 200, 500\}$).

c) Studiați comportamentul lui C_n în funcție de n , tipul repartiției și respectiv valorile parametrilor.

d) Reprezentați grafic evoluția lui C_n și comentați rezultatele.

```

0 # Definim C_n = sup_x |F_n(x) - Phi(x)| * sigma^3 * sqrt(n) / rho
1 # Funcție care calculează C_n pentru un anumit n,
2 # folosind:
3 #   - Fn(x): CDF pentru Z_n (poate fi exact sau prin simulare)
4 #   - rho, sigma: valori teoretice
5 #   - x_grid: grila pe care aproximăm supremumul
6 Cn_from_Fn <- function(n, Fn, rho, sigma, x_grid = make_x_grid()) {
7   sup_val <- sup_diff_numeric(Fn, x_grid) # aprox sup |Fn - Phi|
8   Cn <- sup_val * (sigma^3) * sqrt(n) / rho
9   list(Cn = Cn, sup_val = sup_val)
10 }
11
12 # Varianta universală: folosim Fn prin simulare (merge pt orice distribuție)
13 # rX: funcție de generare (ex: function(nn) rexp(nn, rate=2))
14 Cn_sim <- function(n, rX, mu, sigma, rho, M = 20000, x_grid = make_x_grid()) {
15   Fn_hat <- function(x) Fn_Zn_sim(x, n = n, rX = rX, mu = mu, sigma = sigma, M = M)
16   Cn_from_Fn(n = n, Fn = Fn_hat, rho = rho, sigma = sigma, x_grid = x_grid)
17 }
18
19 # Pentru Binomiala și Poisson putem folosi Fn exact (mai precis și rapid)
20 Cn_binom_exact <- function(n, m, p, x_grid = make_x_grid()) {
21   rho <- rho_binom(m, p)
22   sigma <- sqrt(m * p * (1 - p))
23   Fn <- function(x) Fn_Zn_binom(x, n = n, m = m, p = p)
24   Cn_from_Fn(n = n, Fn = Fn, rho = rho, sigma = sigma, x_grid = x_grid)
25 }
26
27 Cn_pois_exact <- function(n, lambda, x_grid = make_x_grid()) {
28   rho <- rho_pois(lambda)
29   sigma <- sqrt(lambda)
30   Fn <- function(x) Fn_Zn_pois(x, n = n, lambda = lambda)
31   Cn_from_Fn(n = n, Fn = Fn, rho = rho, sigma = sigma, x_grid = x_grid)
32 }
33
34

```



```

1405 set.seed(123)
1406 |
1407 n_values <- c(30, 50, 100, 200, 300, 400, 500)
1408
1409 M_MC <- 20000
1410
1411 # grila pentru aproximarea supremului (cu cuantile normale)
1412 x_grid <- make_x_grid(eps = 1e-4, n_points = 3000)
1413
1414 #####
1415 # Functie generala: estimeaza C_n prin simulare
1416 #####
1417
1418 estimate_Cn_MC <- function(n, rX, mu, sigma, rho, M = 20000, x_grid = x_grid) {
1419
1420   # construim Fn estimat prin CDF empirica a lui Z_n simulat
1421   Fn_hat <- function(x) Fn_Zn_sim(x, n = n, rX = rX, mu = mu, sigma = sigma, M = M)
1422
1423   # sup_x |Fn - Phil aproximat pe grila
1424   sup_val <- sup_diff_numeric(Fn_hat, x_grid)
1425
1426   # definim C_n
1427   Cn <- sup_val * sigma^3 * sqrt(n) / rho
1428
1429   # returnam si sup_val (uneori e util in raport)
1430   list(Cn = Cn, sup_val = sup_val)
1431 }

```

```

1433 #####
1434 # Definim scenariile (distributie + parametri + mu,sigma,rho + generator rX)
1435 # Parametrii ii alegem ca in 7(d) (poti schimba daca vrei).
1436 #####
1437
1438 scenarios_8 <- list(
1439
1440   list(
1441     name = "Binom(m=20,p=0.3)",
1442     mu = 20 * 0.3,
1443     sigma = sqrt(20 * 0.3 * 0.7),
1444     rho = rho_binom(20, 0.3),
1445     rX = function(nn) rbinom(nn, size = 20, prob = 0.3)
1446   ),
1447
1448   # Geometrica in R: suport {0,1,2,...} ("failures")
1449   list(
1450     name = "Geom(p=0.3) [R failures]",
1451     mu = (1 - 0.3) / 0.3,
1452     sigma = sqrt((1 - 0.3) / 0.3^2),
1453     rho = rho_geom(0.3),
1454     rX = function(nn) rgeom(nn, prob = 0.3)
1455   ),
1456
1457   list(
1458     name = "Pois(lambda=5)",
1459     mu = 5,
1460     sigma = sqrt(5),
1461     rho = rho_pois(5),
1462     rX = function(nn) rpois(nn, lambda = 5)
1463   ),
1464
1465   # Uniforma discreta pe {0,1,...,9}
1466   list(
1467     name = "UnifDisc(0..9)",
1468     mu = (0 + 9) / 2,
1469     sigma = sqrt(((10^2) - 1) / 12), # N=10
1470     rho = rho_unif_disc(0, 9),
1471     rX = function(nn) sample(0:9, size = nn, replace = TRUE)
1472   ),
1473
1474   # Uniforma continua U(0,1)
1475   list(
1476     name = "UnifCont(0,1)",
1477     mu = 0.5,
1478     sigma = 1 / sqrt(12),
1479     rho = rho_unif_cont(0, 1),
1480     rX = function(nn) runif(nn, min = 0, max = 1)
1481   ),
1482
1483   # Exponentiala Exp(rate=2)
1484   list(
1485     name = "Exp(lambda=2)",
1486     mu = 1 / 2,
1487     sigma = 1 / 2,
1488     rho = rho_exp(2),
1489     rX = function(nn) rexp(nn, rate = 2)
1490   ),
1491
1492   # Gamma(shape=2, rate=1) (in codul tau: rho_gamma(alpha,beta))
1493   list(
1494     name = "Gamma(alpha=2,beta=1)",
1495     mu = 2,
1496     sigma = sqrt(2),
1497     rho = rho_gamma(2, 1),
1498     rX = function(nn) rgamma(nn, shape = 2, rate = 1)
1499   ),
1500
1501   # Beta(2,5)
1502   list(
1503     name = "Beta(alpha=2,beta=5)",
1504     mu = 2 / 7,
1505     sigma = sqrt(2 * 5 / ((7^2) * 8)),
1506     rho = rho_beta(2, 5),
1507     rX = function(nn) rbeta(nn, shape1 = 2, shape2 = 5)
1508   )
1509 )
1510
1511 #####
1512 # Rulam estimarea pentru fiecare distributie si fiecare n
1513 # si construim dataframe-ul final
1514 #####

```



```

1516 df_Cn <- do.call(rbind, lapply(scenarios_8, function(sc) {
1517
1518   # pentru fiecare n calculam Cn (si optional sup_val)
1519   out <- lapply(n_values, function(n) {
1520     est <- estimate_Cn_MC(n = n, rX = sc$rX, mu = sc$mu, sigma = sc$sigma, rho = sc$rho,
1521                          M = M_MC, x_grid = x_grid)
1522     data.frame(n = n, Cn = est$Cn, sup_val = est$sup_val)
1523   })
1524
1525   tmp <- do.call(rbind, out)
1526   tmp$dist <- sc$name
1527   tmp$mu <- sc$mu
1528   tmp$sigma <- sc$sigma
1529   tmp$rho <- sc$rho
1530   tmp$M <- M_MC
1531   tmp
1532 })
1533
1534 # rearanjam coloanele ca sa fie mai clar
1535 df_Cn <- df_Cn[, c("dist", "n", "Cn", "sup_val", "mu", "sigma", "rho", "M")]
1536
1537 print(df_Cn)
1538

```

```

1      dist      n      Cn      sup_val      mu      sigma      rho      M
2  Binom(m=20,p=0.3) 30 0.09494677 0.027409483 6.0000000 2.0493902 13.609868542 20000
3  Binom(m=20,p=0.3) 50 0.09325829 0.020853668 6.0000000 2.0493902 13.609868542 20000
4  Binom(m=20,p=0.3) 100 0.10011220 0.015829493 6.0000000 2.0493902 13.609868542 20000
5  Binom(m=20,p=0.3) 200 0.09673280 0.010815305 6.0000000 2.0493902 13.609868542 20000
6  Binom(m=20,p=0.3) 300 0.13604703 0.012419623 6.0000000 2.0493902 13.609868542 20000
7  Binom(m=20,p=0.3) 400 0.10942832 0.008651267 6.0000000 2.0493902 13.609868542 20000
8  Binom(m=20,p=0.3) 500 0.15689134 0.011094148 6.0000000 2.0493902 13.609868542 20000
9  Geom(p=0.3) [R failures] 30 0.07884706 0.034976375 2.3333333 2.7888668 52.702740347 20000
10 Geom(p=0.3) [R failures] 50 0.09052278 0.031104468 2.3333333 2.7888668 52.702740347 20000
11 Geom(p=0.3) [R failures] 100 0.07381964 0.017935845 2.3333333 2.7888668 52.702740347 20000
12 Geom(p=0.3) [R failures] 200 0.12189570 0.020942247 2.3333333 2.7888668 52.702740347 20000
13 Geom(p=0.3) [R failures] 300 0.07190034 0.010086029 2.3333333 2.7888668 52.702740347 20000
14 Geom(p=0.3) [R failures] 400 0.13561836 0.016475492 2.3333333 2.7888668 52.702740347 20000
15 Geom(p=0.3) [R failures] 500 0.10516767 0.011427392 2.3333333 2.7888668 52.702740347 20000
16 Poiss(lambda=5) 30 0.06701588 0.019853751 5.0000000 2.2360680 18.141804114 20000
17 Poiss(lambda=5) 50 0.07733496 0.017746649 5.0000000 2.2360680 18.141804114 20000
18 Poiss(lambda=5) 100 0.08291816 0.013454735 5.0000000 2.2360680 18.141804114 20000
19 Poiss(lambda=5) 200 0.07079867 0.008123358 5.0000000 2.2360680 18.141804114 20000
20 Poiss(lambda=5) 300 0.07977529 0.007473658 5.0000000 2.2360680 18.141804114 20000
21 Poiss(lambda=5) 400 0.12972763 0.010525142 5.0000000 2.2360680 18.141804114 20000
22 Poiss(lambda=5) 500 0.16679733 0.012104018 5.0000000 2.2360680 18.141804114 20000
23 UnifDisc(0.9) 30 0.07758475 0.018306736 4.5000000 2.8722813 30.625000000 20000
24 UnifDisc(0.9) 50 0.07235860 0.013225142 4.5000000 2.8722813 30.625000000 20000
25 UnifDisc(0.9) 100 0.11417269 0.014755619 4.5000000 2.8722813 30.625000000 20000
26 UnifDisc(0.9) 200 0.11628473 0.010626809 4.5000000 2.8722813 30.625000000 20000
27 UnifDisc(0.9) 300 0.14497885 0.010817806 4.5000000 2.8722813 30.625000000 20000
28 UnifDisc(0.9) 400 0.09512650 0.006147049 4.5000000 2.8722813 30.625000000 20000
29 UnifDisc(0.9) 500 0.17694276 0.010226876 4.5000000 2.8722813 30.625000000 20000
30 UnifCont(0,1) 30 0.03841797 0.009111621 0.5000000 0.2886751 0.031250000 20000
31 UnifCont(0,1) 50 0.02428653 0.004461721 0.5000000 0.2886751 0.031250000 20000
32 UnifCont(0,1) 100 0.06739437 0.008754785 0.5000000 0.2886751 0.031250000 20000
33 UnifCont(0,1) 200 0.07825045 0.007187763 0.5000000 0.2886751 0.031250000 20000
34 UnifCont(0,1) 300 0.11878426 0.008908820 0.5000000 0.2886751 0.031250000 20000
35 UnifCont(0,1) 400 0.08159345 0.005299650 0.5000000 0.2886751 0.031250000 20000
36 UnifCont(0,1) 500 0.10215688 0.005934778 0.5000000 0.2886751 0.031250000 20000
37 Exp(lambda=2) 30 0.06412652 0.028269223 0.5000000 0.5000000 0.301819162 20000
38 Exp(lambda=2) 50 0.06179783 0.021102067 0.5000000 0.5000000 0.301819162 20000
39 Exp(lambda=2) 100 0.05521896 0.013332911 0.5000000 0.5000000 0.301819162 20000
40 Exp(lambda=2) 200 0.05369279 0.009167222 0.5000000 0.5000000 0.301819162 20000
41 Exp(lambda=2) 300 0.13967604 0.019471440 0.5000000 0.5000000 0.301819162 20000
42 Exp(lambda=2) 400 0.08629759 0.010418506 0.5000000 0.5000000 0.301819162 20000
43 Exp(lambda=2) 500 0.09091079 0.009816739 0.5000000 0.5000000 0.301819162 20000
44 Gamma(alpha=2,beta=1) 30 0.04855189 0.018002201 2.0000000 1.4142136 5.744140393 20000
45 Gamma(alpha=2,beta=1) 50 0.07240189 0.020794331 2.0000000 1.4142136 5.744140393 20000
46 Gamma(alpha=2,beta=1) 100 0.06342654 0.012881044 2.0000000 1.4142136 5.744140393 20000
47 Gamma(alpha=2,beta=1) 200 0.06886633 0.009889446 2.0000000 1.4142136 5.744140393 20000
48 Gamma(alpha=2,beta=1) 300 0.08961060 0.010507002 2.0000000 1.4142136 5.744140393 20000
49 Gamma(alpha=2,beta=1) 400 0.07364983 0.007478626 2.0000000 1.4142136 5.744140393 20000
50 Gamma(alpha=2,beta=1) 500 0.05361948 0.004869873 2.0000000 1.4142136 5.744140393 20000
51 Beta(alpha=2,beta=5) 30 0.02012406 0.005706786 0.2857143 0.1597191 0.006328599 20000
52 Beta(alpha=2,beta=5) 50 0.06113693 0.013429360 0.2857143 0.1597191 0.006328599 20000
53 Beta(alpha=2,beta=5) 100 0.04820898 0.007487979 0.2857143 0.1597191 0.006328599 20000
54 Beta(alpha=2,beta=5) 200 0.04906617 0.005388946 0.2857143 0.1597191 0.006328599 20000
55 Beta(alpha=2,beta=5) 300 0.09726993 0.008722774 0.2857143 0.1597191 0.006328599 20000
56 Beta(alpha=2,beta=5) 400 0.11561638 0.008978960 0.2857143 0.1597191 0.006328599 20000
57 Beta(alpha=2,beta=5) 500 0.09018540 0.006264522 0.2857143 0.1597191 0.006328599 20000
>

```

```

#####
# EXERCITIUL 5(c)
# Analizam cum variaza C_n in functie de n si intre distributii.
#####

```

```

# 1) Rezumat: pentru fiecare distributie, luam valorile Cn la n mari
# (de ex. n >= 100) si calculam media si maximul.
df_large_n <- subset(df_Cn, n >= 100)

```

```

summary_Cn <- aggregate(Cn ~ dist, data = df_large_n,
                        FUN = function(v) c(mean = mean(v), max = max(v), min = min(v)))

```

```

# aggregate intoarce o coloana "Cn" cu vectori; o "despachetam"
summary_Cn <- do.call(data.frame, summary_Cn)
colnames(summary_Cn) <- c("dist", "Cn_mean", "Cn_max", "Cn_min")

print(summary_Cn)

```

```

# 2) "C recomandat" (empiric): luam maximul lui Cn pe n mari, peste toate distributiile
C_recommended <- max(df_large_n$Cn)
C_recommended

```

```

> print(summary_Cn)
      dist      Cn_mean      Cn_max      Cn_min
1  Beta(alpha=2,beta=5) 0.08006937 0.1156164 0.04820898
2  Binom(m=20,p=0.3) 0.11984234 0.1568913 0.09673280
3  Exp(lambda=2) 0.08515923 0.1396760 0.05369279
4  Gamma(alpha=2,beta=1) 0.06983456 0.0896106 0.05361948
5  Geom(p=0.3) [R failures] 0.10168034 0.1356184 0.07190034
6  Poiss(lambda=5) 0.10600342 0.1667973 0.07079867
7  UnifCont(0,1) 0.08963588 0.1187843 0.06739437
8  UnifDisc(0.9) 0.12950111 0.1769428 0.09512650

```

```

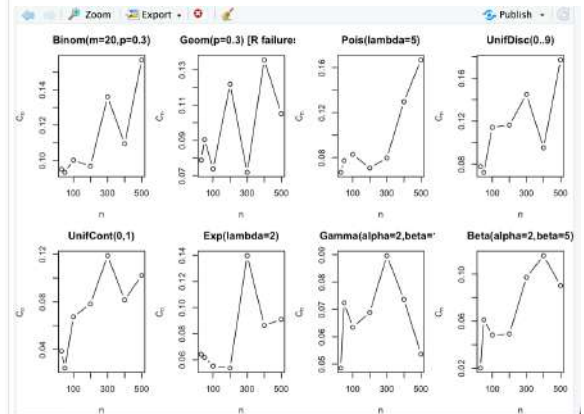
>
> # 2) "C recomandat" (empiric): luam maximul lui Cn pe n mari, peste toate distributiile
> C_recommended <- max(df_large_n$Cn)
> C_recommended
[1] 0.1769428
~

```

```

1564 # EXERCITIUL 5(d)
1565 # Prezentare rezultate: tabele + grafice pentru C_n
1566 #####
1567
1568 # Presupunem ca ai df_Cn din 8(b)
1569 # df_Cn are coloane: dist, n, Cn, sup_val, mu, sigma, rho, M
1570
1571 #####
1572 # 1) Tabel "wide": cate o coloana pentru fiecare n
1573 #####
1574 # Convertim df_Cn din "long" in "wide" ca sa fie usor de pus in raport
1575
1576 df_wide <- reshape(df_Cn[, c("dist", "n", "Cn")],
1577                    timevar = "n", idvar = "dist", direction = "wide")
1578
1579 # Ca sa arate frumos: ordonam coloanele crescator dupa n
1580 # (reshape pune Cn.30, Cn.50, etc.)
1581 df_wide <- df_wide[, c("dist", sort(setdiff(names(df_wide), "dist")))]
1582
1583 print(df_wide)
1584
1585 #####
1586 # 2) Rezumat pe distributie (ex: media/max pe n mari)
1587 #####
1588 df_large_n <- subset(df_Cn, n >= 100)
1589
1590 summary_Cn <- aggregate(Cn ~ dist, data = df_large_n,
1591                          FUN = function(v) c(mean = mean(v), max = max(v), min = min(v)))
1592 summary_Cn <- do.call(data.frame, summary_Cn)
1593 colnames(summary_Cn) <- c("dist", "Cn_mean_n=>100", "Cn_max_n=>100", "Cn_min_n=>100")
1594
1595 print(summary_Cn)
1596
1597 #####
1598 # 3) Grafice separate (2x4): Cn vs n pentru fiecare distributie
1599 #####
1600 dists <- unique(df_Cn$dist)
1601
1602 par(mfrow = c(2, 4))
1603 for (d in dists) {
1604   tmp <- subset(df_Cn, dist == d)
1605   plot(tmp$n, tmp$Cn, type = "b",
1606        xlab = "n", ylab = expression(C[n]),
1607        main = d)
1608 }
1609 par(mfrow = c(1, 1))
1610
1611 #####
1612 # 4) Grafic comparativ: toate distributiile pe aceeaasi figura
1613 #####
1614 # Facem un plot gol, apoi adaugam linii pentru fiecare dist.
1615
1616 plot(range(df_Cn$n), range(df_Cn$Cn),
1617      type = "n", xlab = "n", ylab = expression(C[n]),
1618      main = expression(paste("Comparatie", C[n], "intre distributii")))
1619
1620 for (d in dists) {
1621   tmp <- subset(df_Cn, dist == d)
1622   lines(tmp$n, tmp$Cn, type = "b")
1623 }
1624
1625 legend("topright", legend = dists, cex = 0.6, bty = "n")

```



1540:1 (Untitled) :

```

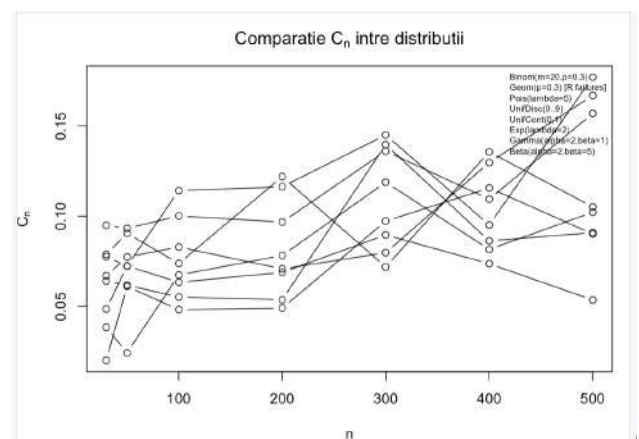
> print(df_wide)
      dist  Cn.100  Cn.200  Cn.30  Cn.300  Cn.400  Cn.50  Cn.500
1 Binom(m=20,p=0.3) 0.10011220 0.09673280 0.09494677 0.13604703 0.10942832 0.09325829 0.15689134
8 Geom(p=0.3) [R failures] 0.07381964 0.12189570 0.07884706 0.07190034 0.13561836 0.09052278 0.10516767
15 Pois(lambda=5) 0.08291816 0.07079867 0.06701588 0.07977529 0.12972763 0.07733496 0.16679733
22 UnifDisc(0.9) 0.11417269 0.11628473 0.07758475 0.14497885 0.09512650 0.07235860 0.17694276
29 UnifCont(0,1) 0.06739437 0.07825045 0.03841797 0.11878426 0.08159345 0.02428653 0.10215688
36 Exp(lambda=2) 0.05521896 0.05369279 0.06412652 0.13967604 0.08629759 0.06179783 0.09091079
43 Gamma(alpha=2,beta=1) 0.06342654 0.06886633 0.04855189 0.08961060 0.07364983 0.07240189 0.05361948
50 Beta(alpha=2,beta=5) 0.04820898 0.04906617 0.02012406 0.09726993 0.11561638 0.06113693 0.09018540

```

```

> print(summary_Cn)
      dist Cn_mean_n=>100 Cn_max_n=>100 Cn_min_n=>100
1 Beta(alpha=2,beta=5) 0.08006937 0.1156164 0.04820898
2 Binom(m=20,p=0.3) 0.11984234 0.1568913 0.09673280
3 Exp(lambda=2) 0.08515923 0.1396760 0.05369279
4 Gamma(alpha=2,beta=1) 0.06983456 0.0896106 0.05361948
5 Geom(p=0.3) [R failures] 0.10168034 0.1356184 0.07190034
6 Pois(lambda=5) 0.10600342 0.1667973 0.07079867
7 UnifCont(0,1) 0.08963588 0.1187843 0.06739437
8 UnifDisc(0.9) 0.12950111 0.1769428 0.09512650

```



Concluzii

a) Rolul familiei exponențiale în estimarea parametrilor

Familia exponențială reunește multe distribuții importante într-o formă matematică comună. Acest lucru permite estimarea parametrilor folosind doar câteva mărimi simple calculate din date, cum ar fi sume sau medii. În consecință, metodele de estimare devin mai ușor de aplicat și mai eficiente, fără a pierde din rigoarea teoretică.

b) Influența momentului de ordin 3 asupra vitezei de convergență în teorema limitei centrale

Momentul de ordin 3 măsoară gradul de asimetrie al unei distribuții. Rezultatele teoretice arată că distribuțiile mai asimetrice converg mai lent către distribuția normală. Simulările numerice confirmă acest fapt, evidențiind că este necesar un număr mai mare de observații pentru ca aproximarea normală să fie bună atunci când distribuția inițială este dezechilibrată.

c) Semnificația practică a constantei din inegalitatea Berry–Esseen

Constanta din inegalitatea Berry–Esseen oferă o limită superioară a erorii dintre distribuția reală a sumei normalizate și distribuția normală. Din punct de vedere practic, aceasta indică cât de precisă poate fi aproximarea normală pentru un eșantion finit. Valorile estimate numeric arată că această eroare scade odată cu creșterea volumului de date și rămâne controlată pentru distribuțiile analizate.

d) Diferențe între repartiții în ceea ce privește acuratețea aproximării normale

Acuratețea aproximării normale depinde atât de dimensiunea eșantionului, cât și de forma distribuției inițiale. Distribuțiile mai simetrice sunt mai bine approximate pentru valori moderate ale lui n , în timp ce distribuțiile asimetrice sau discrete necesită eșantioane mai mari. Aceste diferențe sunt observate și în rezultatele numerice obținute.

e) Concluzie generală privind familia exponențială

Rezultatele teoretice și numerice obținute arată că familia exponențială oferă un cadru coerent și eficient atât pentru estimarea parametrilor, cât și pentru studiul comportamentului asimptotic. Această familie permite aplicarea unitară a teoremei limitei centrale, iar diferențele dintre distribuții apar în principal în viteza de convergență, nu în validitatea aproximării.